

An Agentic Bug-Hunt in SymPy

Xinjie He Hyunsik Chae Ernest K. Ryu

Abstract

The open-source computer algebra system SymPy is a core piece of mathematical infrastructure, widely trusted as an oracle for symbolic computation. In this work, we present an agentic harness, built on current coding agents, that audits such software: it searches for candidate correctness bugs, independently verifies them, checks for public duplicates, diagnoses likely source-level mechanisms, and emits reproducible artifact bundles. Applied to SymPy, the harness finds many silent correctness failures in routine algebra and calculus. We present a curated catalog of representative bugs, each accompanied by code that reproduces the failure and a likely explanation of the root cause.

1 Introduction

The open-source computer algebra system SymPy [1] is a core piece of mathematical infrastructure, used across a wide range of disciplines and applications—both on its own and as a dependency of downstream scientific tools such as SageMath [2], PyDy [3], and SfePy [4]. While it is understood that any large software library will inevitably contain bugs, many users routinely treat such systems as trusted tools; the name recognition and widespread use of SymPy, in particular, can give the impression that it is thoroughly “battle-tested” and reliable.

In this work, we show that an agentic LLM harness reveals that SymPy has many strikingly simple failures in which it silently returns a plausible, well-formed expression that is mathematically incorrect. Our goal is twofold: (i) to contribute to the open-source SymPy project by identifying such failures and turning them into reproducible test cases for debugging, and (ii) to demonstrate that modern agentic LLM tools are remarkably effective at this kind of task.

Agents and harnesses. By *harness*, we mean the setup that determines how a language model is used: the prompts it receives, the tools it may use, and how tool results determine the next step. We call the combination of a language model and such a harness an *agent*. An agent can carry out a multi-step task by running tools, reading their outputs, and deciding how to proceed [12]. Modern coding agents, such as OpenAI Codex [13] and Claude Code [17], are instances of this paradigm.

The notion of a harness can also be applied at a higher level: a harness may wrap an already packaged agent, rather than only a raw language model. Our harness operates in this way. It uses an existing coding agent, while fixing the higher-level workflow of searching for candidate bugs, verifying them, checking for duplicates, and packaging reproducers.

Related work. Computer algebra systems have long been known to have bugs [5], but testing them is hard because checking an answer can require solving the original problem again [6]. Before LLMs, one common workaround was to check the “contract” between the input and output [7]: for instance, differentiating a proposed integral and checking whether it recovers the integrand. With LLMs becoming increasingly capable at mathematics, we can also use them as judges of correctness when the underlying math is simple. This is the approach we propose here. In our manual checks of the 63 bugs released with this paper, we found one case in which the LLM-derived ground truth was imprecise, although the underlying SymPy output was still incorrect; the remaining 62 had correct ground truths, and none of the inspected candidates was spurious.

Recent work has used language models for software testing and repair, from fuzzing deep learning libraries and other software systems [8, 9] to resolving real software issues end to end [10]. These systems mainly target crashes, code coverage, or test-suite failures. Our focus is different: the program does not crash, but returns a plausible mathematical answer that is mathematically incorrect.

Table 1: Example bugs from the catalog (Section 3); the case numbers match the cards there.

Case	SymPy’s failure
1	Solving $\operatorname{asin}(x) = \pi$ over $\mathbb{C}$, returns $x = 0$, although $\operatorname{asin}(0) = 0$ and π lies outside the principal range.
2	Solving $\sqrt{x^2} = x$ over $\mathbb{C}$, claims that <i>every</i> complex number is a solution (e.g. $x = -1$ fails).
3	Solving $\sec(x) = 0$ over $\mathbb{C}$, returns $\{\text{nan}\}$, although $\sec$ is never zero and $\sec(\text{nan}) = \text{nan}$.
4	Returns an antiderivative of $\log(x)/(x^2 - 1)$ whose derivative differs from the integrand by an erroneous $-\pi i/2$.
5	Returns an antiderivative of $\sqrt{x^2 + 1}/x$ with the wrong sign on negative reals (gives $+\sqrt{5}/2$ at $x = -2$ instead of $-\sqrt{5}/2$).
6	The left-hand limit at 0 of $\text{Piecewise}((0, x < 0), (1, \text{True}))$ gives the right-side value 1 instead of 0.
7	Solving $f' = \sqrt{f^2}$, returns $f = C_1 e^{-x}$ with C_1 unrestricted; for $C_1 > 0$, this gives $f' < 0$ although $\sqrt{f^2} \geq 0$.

continued on next page

Table 1 (continued)

Case	SymPy's failure
8	Returns 0 for $\sum_{k=0}^n (-1)^k k \binom{n}{k}$, dropping the $n = 1$ exception (which is -1).
9	Returns 0 for $\prod_{n=0}^2 \sin(\pi n)/(n-1)$, although the factor at $n = 1$ is $0/0$ so the product is undefined.
10	Solving the principal cube-root inequality $x^{1/3} \leq 1$ over $\mathbb{R}$, returns $(-\infty, 1]$, including negative x where the principal root is non-real; the answer is $[0, 1]$.
11	Reports the real range of $\sqrt{x^2}$ as $\emptyset$ instead of $[0, \infty)$.
12	Returns e^{-t} for the inverse Fourier transform of a Lorentzian instead of the even $e^{- t }$.
13	Returns $(1, 1)$ for a rational system, although $x = 1$ makes one equation undefined; the answer is $\emptyset$.
14	For a Rayleigh variable $R \geq 0$, reports $\text{cdf}(R)(-1) = 1 - e^{-1/2} > 0$.
15	Returns the empty set for the $p = 1$ quantile of $\text{Uniform}(0, 1)$ instead of $\{1\}$.
16	Returns e^{-as}/s for the Laplace transform of $\text{Heaviside}(a - t)$; the transform is $(1 - e^{-as})/s$ for $a > 0$ and 0 for $a < 0$.
17	Returns -1 for $\sum_{k=0}^{\infty} 1/(k(k-1))$, although the $k = 0, 1$ terms are undefined.
18	Evaluates $\sinh(i\pi + i(\pi - 1))$ to 0 instead of $-i \sin(1)$.
19	Returns the identically zero sequence (every term is 0) for a recurrence with $y(0) = y(1) = 1$, violating both initial conditions.
20	The density of $U \sim \text{Uniform}(0, 1)$ conditioned on $U > 1/2$ keeps support $[0, 1]$, giving total mass 2 instead of 1.

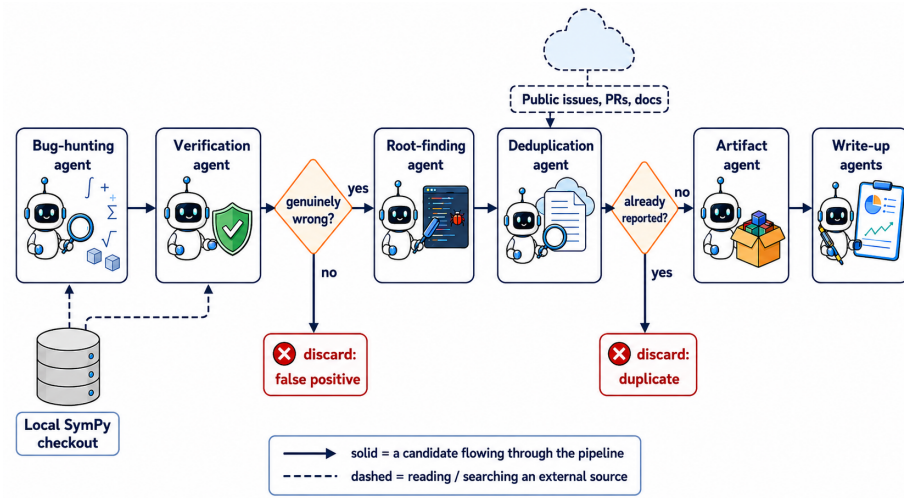


Figure 1: Illustration of the SymPy bug-hunting harness. (Image generated with ChatGPT Images 2.0)

2 Harness design

Our agentic harness orchestrates six specialized sub-agents as illustrated in Figure 1 and described below.

The **bug-hunting agent** searches a pinned SymPy checkout for candidate bugs, with instructions to focus on mathematical correctness failures over other software issues. The **verification agent** then independently checks whether each candidate is a genuine bug rather than a hallucination. We found it important that this division of labor reduces the risk that an agent reports a nonexistent bug and then rationalizes its own claim.

The **root-finding agent** inspects the SymPy source to identify the likely cause of each verified candidate. This analysis is best-effort: it points to a probable source of the failure, with no guarantee that patching the identified code path would eliminate the bug or that it is the “correct” fundamental root cause.

The **deduplication agent** searches public issues, pull requests, release notes, and online discussions for prior reports of the failure. Many bugs are already known to the community but remain unfixed, and re-reporting flagged issues adds little value. We therefore retain a candidate only when fixing existing known issues would not automatically resolve it.

The **artifact agent** prepares a script that reproduces each confirmed non-duplicate bug (`reproduce_bug.py`), a best-effort source-level diagnosis (`root_cause.md`), a deduplication note (`dedup_report.md`), additional related bugs (`related_bugs.py`), a draft pull request following SymPy’s testing

conventions, and finally a `README.md` summarizing all the information above (Figure 2).

Finally, the **write-up agent** turns the accepted artifacts into a $\text{\LaTeX}$ technical report, which we (humans) then polished by hand and incorporated into the appendix of this paper.

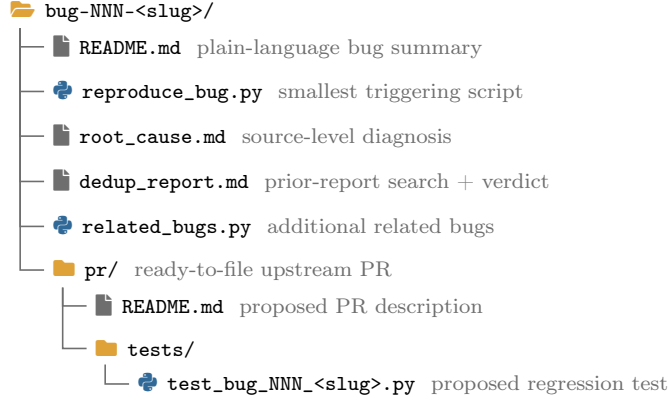


Figure 2: What the artifact agent produces for each identified bug

Configuration and cost. All six agents are non-interactive `codex exec` calls to OpenAI’s Codex CLI running GPT-5.6 Sol (medium reasoning effort) [14], used entirely through a ChatGPT subscription rather than the metered API, so a full run needs no API key. Appendix A gives the exact configuration. A default run of 10 confirmed bugs uses only 1% of a weekly ChatGPT Pro subscription and no metered API calls. Converted to pay-as-you-go API list prices, this works out to only about \$2 per confirmed bug—an imputed per-token estimate for reference, since the actual subscription bills a flat recurring fee rather than per token. The harness itself—agent prompts, configuration, and orchestration—is released in the companion repository,¹ and can be run either through `codex exec` or interactively via Codex’s GUI app; both use the same ChatGPT subscription, so the cost is comparable.

3 Bug catalog

Again, our harness is fully autonomous and includes verification mechanisms. During development and across several runs, it produced roughly 100 distinct candidate bugs. We (humans) manually checked 63 of them, and all 63 were confirmed as genuine SymPy failures on the latest stable SymPy release (version 1.14.0). On the most recent development branch as of publishing this report, at commit 15e21aca70, 62 of the 63 still reproduced; the remaining bug,

¹<https://github.com/xinjiegit/agent-ic-bug-hunt-sympy>

bug 041 in the companion repository and not one of the 20 cataloged below, appears to have been fixed on the development branch. In one artifact-only case, bug 43, the harness correctly identified that SymPy’s range computation for $\cos(x)/\cot(x)$ returning $\{0\}$ is wrong, but its proposed ground truth, $(-1, 1)$, was also imprecise because 0 is unattainable. Thus, none of the inspected candidates turned out to be spurious, but one of the released artifacts contained a ground-truth mistake. This evidence suggests that the harness has a low false-positive rate while underscoring the need for human verification. Nevertheless, in this paper, we report only the bugs that we have manually verified.

Among the human verified bugs, we selected what we considered the 20 most interesting ones for this report; the remaining 43 are included in the same companion repository. The candidate bugs that have not yet been manually inspected are available upon request; however, we do not plan to post them publicly before human verification.

The cards below give compact view of the 20 bugs; full analyses are presented in Appendix A. The artifacts for each bug are also available in the same repository.

Case 1: solveset accepts $\text{asin}(x) = \pi$

Task. Solve the principal inverse-sine equation over the complex numbers.

$$\text{asin}(x) = \pi, \quad x \in \mathbb{C}.$$

SymPy’s answer.

$$\text{solveset}(\text{asin}(x) = \pi, x, \mathbb{C}) = \{0\}.$$

Correct answer.

$$\emptyset.$$

Explanation. SymPy evaluates $\text{asin}(0) = 0$, so the returned candidate is not a solution; since π lies outside the principal range of asin , the equation is unsatisfiable.

Likely root cause. In `sympy/solvers/solveset.py`, `_invert_complex` appears to apply `sin` as a global inverse of principal `asin`, without carrying the required range condition, and the finite candidate is not rejected by a residual check. This is a new instance of a known failure mode.

Artifact (GitHub).

https://github.com/xinjiegit/agent-ic-bug-hunt-sympy/tree/main/released_bugs/bug-001-solveset-accepts-asin-x-pi/

Case 2: solveset claims $\sqrt{x^2} = x$ for all complexes

Task. Solve the principal-square-root equation over the complex numbers:

$$\sqrt{x^2} = x.$$

SymPy's answer.

$$\text{solveset}(\sqrt{x^2} = x, x, \mathbb{C}) = \mathbb{C}.$$

Correct answer.

$$\{z : \text{Re}(z) > 0\} \cup \{it : t \in \mathbb{R}, t \geq 0\},$$

in particular not $\mathbb{C}$.

Explanation. The set contains $x = -1$, yet $\sqrt{(-1)^2} = 1 \neq -1$. Squaring treats $\sqrt{x^2} = x$ as branch-insensitive, dropping the principal-root branch.

Likely root cause. In `sympy/solvers/solveset.py`, `_solve_radical` accepts the non-finite result produced after `unrad` turns the equation into $0 = 0$, without validating it against the original equation. This is a new instance of a known failure mode.

Artifact (GitHub). https://github.com/xinjiegit/agent-ic-bug-hunt-sympy/tree/main/released_bugs/bug-002-solveset-claims-sqrt-x-2-x-for-all-complexes/

Case 3: solveset gives nan as a secant zero

Task. Solve the secant equation over the complex domain:

$$\sec(x) = 0.$$

SymPy's answer.

$$\text{solveset}(\sec(x), x, \mathbb{C}) = \{\text{nan}\}.$$

Correct answer.

$$\emptyset.$$

Explanation. For finite complex z , $\sec(z) = 1/\cos(z)$ is never zero. The reported candidate is not a complex solution; $\sec(\text{nan}) = \text{nan}$, not 0.

Likely root cause. The diagnosis traces the result to `_invert_trig_hyp_complex` in `sympy/solvers/solveset.py`, which applies `asec(0)` while inverting $\sec(x) = 0$ and lacks a reciprocal-trig range guard excluding zero. This is a new instance of a known failure mode.

Artifact (GitHub). https://github.com/xinjiegit/agent-ic-bug-hunt-sympy/tree/main/released_bugs/bug-003-solveset-gives-nan-as-a-secant-zero/

Case 4: real antiderivative gains an imaginary term

Task. Compute an indefinite integral and check that it differentiates back:

$$\int \frac{\log x}{x^2 - 1} dx.$$

SymPy's answer. *SymPy* returns an expression F for which

$$F'(2) = \frac{\log 2}{3} - \frac{\pi i}{2}.$$

Correct answer. A valid local antiderivative must satisfy

$$F'(2) = \frac{\log 2}{3}$$

Explanation. The integrand $\log(x)/(x^2 - 1)$ is real and analytic near $x = 2$. An additive integration constant would vanish under differentiation, so the nonzero residual $-\pi i/2$ is an erroneous branch term in the derivative, not an equivalent antiderivative representation.

Likely root cause. The diagnosis locates the failure in `sympy/integrals/meijerint.py`: integration by parts delegates $\int \log(x - 1)/x dx$ to Meijer integration, whose shifted-log rewrite selects a branch that differentiates to $\log(x - 1)/x + i\pi/x$. This is a new instance of a known failure mode.

Artifact (GitHub). https://github.com/xinjiegit/agent-ic-bug-hunt-sympy/tree/main/released_bugs/bug-004-real-antiderivative-gains-an-imaginary-term/

Case 5: antiderivative has the wrong sign for negative x

Task. Compute an indefinite integral:

$$\int \frac{\sqrt{x^2 + 1}}{x} dx.$$

SymPy's answer. SymPy returns

$$\frac{x}{\sqrt{1 + x^{-2}}} - \operatorname{asinh}\left(\frac{1}{x}\right) + \frac{1}{x\sqrt{1 + x^{-2}}}.$$

Correct answer. The returned formula differentiates to

$$\sqrt{1 + x^{-2}},$$

which differs from the original integrand $\sqrt{x^2 + 1}/x$ on the negative real interval.

Explanation. An antiderivative must differentiate back to the integrand. Here, substituting $x = -2$ after differentiation gives $+\sqrt{5}/2$, while the original integrand gives $-\sqrt{5}/2$.

Likely root cause. The diagnosis locates the branch loss in `sympy/integrals/meijerint.py`, where the Meijer-G indefinite integration path rewrites through an x^2 argument and then denests powers, producing a positive-branch formula without the needed $\operatorname{sign}(x)$ factor. This is a new instance of a known failure mode.

Artifact (GitHub). https://github.com/xinjiegit/agent-ic-bug-hunt-sympy/tree/main/released_bugs/bug-005-antiderivative-has-the-wrong-sign-for-negative-x/

Case 6: left limit gives the right-side value

Task. Evaluate $\lim_{x \rightarrow 0^-} p(x)$ of the discontinuous function

$$p(x) = \begin{cases} 0, & x < 0, \\ 1, & x \geq 0. \end{cases}$$

SymPy's answer. 1.

Correct answer. 0.

Explanation. For every sufficiently small negative x , the condition $x < 0$ is true, so the Piecewise expression is identically 0 on the left side of the jump. SymPy returns 1, the value of the right branch.

Likely root cause. The diagnosis points to `Piecewise._eval_as_leading_term` in `sympy/functions/elementary/piecewise.py`: it receives the approach direction `cdir` but substitutes the boundary value into branch conditions instead of using the requested side. This is a new instance of a known failure mode.

Artifact (GitHub). https://github.com/xinjiegit/agent-ic-bug-hunt-sympy/tree/main/released_bugs/bug-006-left-limit-gives-the-right-side-value/

Case 7: ODE solver loses signs in $f' = |f|$

Task. Solve the first-order ODE

$$f'(x) = \sqrt{f(x)^2}.$$

SymPy's answer.

$$f(x) = C_1 e^{-x} \quad \text{or} \quad f(x) = C_1 e^x, \quad C_1 \in \mathbb{R}.$$

Correct answer.

$$f(x) = 0, \quad f(x) = C e^x \ (C > 0), \quad f(x) = C e^{-x} \ (C < 0).$$

Explanation. For real-valued solutions, the equation is $f' = |f|$. A nonzero solution cannot change sign, so a negative solution satisfies $f' = -f$ and a positive solution satisfies $f' = f$.

Likely root cause. In `sympy/solvers/ode/ode.py`, `odesimp` appears to force a branch-sensitive implicit separable solution through `solve(..., force=True)` and then returns explicit branches without the required sign condition. This is a new instance of a known failure mode.

Artifact (GitHub). [https:](https://github.com/xinjiegit/agent-ic-bug-hunt-sympy/tree/main/released_bugs/bug-007-ode-solver-loses-signs-in-f-abs-f/)

[//github.com/xinjiegit/agent-ic-bug-hunt-sympy/tree/main/
released_bugs/bug-007-ode-solver-loses-signs-in-f-abs-f/](https://github.com/xinjiegit/agent-ic-bug-hunt-sympy/tree/main/released_bugs/bug-007-ode-solver-loses-signs-in-f-abs-f/)

Case 8: k -weighted binomial sum drops the $n = 1$ exception

Task. Evaluate the k -weighted alternating binomial sum for a nonnegative integer n .

$$\sum_{k=0}^n (-1)^k k \binom{n}{k}$$

SymPy's answer.

$$\text{summation}\left((-1)^k k \binom{n}{k}, (k, 0, n)\right) = 0$$

Correct answer.

$$\begin{cases} -1, & n = 1, \\ 0, & n \in \mathbb{Z}_{\geq 0}, n \neq 1. \end{cases}$$

Explanation. At $n = 1$, the finite sum is $0 \binom{1}{0} - 1 \binom{1}{1} = -1$, but SymPy's result is the constant function 0 for all n .

Likely root cause. The diagnosis traces the result to the Gosper finite-sum branch in `sympy/concrete/gosper.py`: the certificate $-(k-1)/(n-1)$ has a pole at $n = 1$, but the endpoint formula simplifies to 0 before that excluded parameter is preserved. This is a new instance of a known failure mode.

Artifact (GitHub). https://github.com/xinjiegit/agent-ic-bug-hunt-sympy/tree/main/released_bugs/bug-008-k-weighted-binomial-sum-drops-the-n-1-exception/

Case 9: finite product masks an undefined factor

Task. Evaluate the finite product

$$\prod_{n=0}^2 \frac{\sin(\pi n)}{n-1}.$$

SymPy's answer. `product(sin( $\pi n$ )/( $n-1$ ), ( $n, 0, 2$ )) = 0`.

Correct answer. The product is nan, or should remain unevaluated.

Explanation. At $n = 1$, the factor is $\sin(\pi)/(1-1) = 0/0$, so the product contains an undefined term. Hence the finite product is not equal to 0, even though the factors at $n = 0$ and $n = 2$ vanish.

Likely root cause. In `Product.doit`, SymPy replaces the index by an integer-assumption dummy before evaluating the finite product, so $\sin(\pi n)$ collapses to 0 and masks the denominator singularity at $n = 1$. This is a new failure mode with no public precedent.

Artifact (GitHub). https://github.com/xinjiegit/agent-ic-bug-hunt-sympy/tree/main/released_bugs/bug-009-finite-product-masks-an-undefined-factor/

Case 10: principal cube-root inequality includes negatives

Task. Solve the inequality over real x as represented in SymPy by `x**(S(1)/3) <= 1`.

$$x^{1/3} \leq 1$$

SymPy's answer.

$$(-\infty, 1].$$

Correct answer.

$$[0, 1].$$

Explanation. The expression `x**(S(1)/3)` is a principal `Pow`, not SymPy's real-root operation. Thus, unlike the real cube root, SymPy's principal value at $x = -1$ is

$$(-1)^{1/3} = \frac{1}{2} + \frac{\sqrt{3}}{2}i,$$

so the ordered real comparison with 1 is not satisfied there. On the real-valued domain $x \geq 0$, the principal cube root is monotone and $x^{1/3} \leq 1$ gives $0 \leq x \leq 1$.

Likely root cause. The diagnosis points to `continuous_domain` treating odd-denominator fractional powers as real-valued over all real bases, which matches `real_root` semantics rather than principal `Pow` semantics; the inequality solver then samples and accepts an interval containing negative inputs. Deduplication frames this as a new instance of a known failure mode.

Artifact (GitHub). https://github.com/xinjiegit/agent-ic-bug-hunt-sympy/tree/main/released_bugs/bug-010-principal-cube-root-inequality-includes-negatives/

Case 11: `function_range` gives `EmptySet` for $\sqrt{x^2}$

Task. Compute the real range of the square root of a square.

$$\text{function_range}\left(\sqrt{x^2}, x, \mathbb{R}\right)$$

SymPy's answer.

$$\text{function_range}\left(\sqrt{x^2}, x, \mathbb{R}\right) = \emptyset$$

Correct answer.

$$[0, \infty)$$

Explanation. For real x , $\sqrt{x^2} = |x|$, whose range is $[0, \infty)$, not $\emptyset$.

Likely root cause. The diagnosis identifies `function_range` in `sympy/calculus/util.py`: it samples endpoint limits and solutions of $f'(x) = 0$, but misses nondifferentiable interior extrema such as the cusp of $\sqrt{x^2}$ at $x = 0$. Deduplication classified this as a new failure mode with no public precedent.

Artifact (GitHub). https://github.com/xinjiegit/agent-ic-bug-hunt-sympy/tree/main/released_bugs/bug-011-function-range-gives-emptyset-for-sqrt-x-2/

Case 12: inverse Fourier transform drops $|t|$

Task. Compute the inverse Fourier transform of the Lorentzian

$$F(w) = \frac{2}{1 + 4\pi^2 w^2}.$$

SymPy's answer.

$$\text{inverse_fourier_transform}(F(w), w, t) = e^{-t}.$$

Correct answer.

$$\text{inverse_fourier_transform}(F(w), w, t) = e^{-|t|}.$$

Explanation. The input $F(w)$ is real and even in w , so its inverse Fourier transform should be real and even in t . SymPy returns e^{-t} , which is not even: for example, at $t = -1$ it gives e , while the expected value is e^{-1} . The missing absolute value changes the answer on the negative half-line.

Likely root cause. In `sympy/integrals/transforms.py`, `_fourier_transform` appears to select the first branch of a conditional `Piecewise` integral, then the public inverse transform drops the condition under the default `noconds=True`. This is a new instance of a known failure mode.

Artifact (GitHub). https://github.com/xinjiegit/agent-ic-bug-hunt-sympy/tree/main/released_bugs/bug-012-inverse-fourier-transform-drops-abs-t/

Case 13: nonlinsolve accepts an undefined point as a solution

Task. Solve the rational system with SymPy's `nonlinsolve`.

$$\left\{ \frac{xy - x}{x - 1} - y = 0, \quad y - 1 = 0 \right\}$$

SymPy's answer.

$$\text{nonlinsolve}\left(\left[\frac{xy - x}{x - 1} - y, y - 1\right], [x, y]\right) = \{(1, 1)\}.$$

Correct answer.

$$\emptyset$$

Explanation. Substituting $(1, 1)$ makes the original denominator $x - 1$ equal to 0, so the first rational equation is undefined there. Clearing the denominator yields the polynomial condition $y - x = 0$, but that step is valid only when $x \neq 1$; its apparent solution $(1, 1)$ is therefore removed by the original domain restriction.

Likely root cause. In `sympy/solvers/solveset.py`, `nonlinsolve` records denominators before converting rational equations to numerators, but its no-remaining-equations polynomial success path returns `FiniteSet(*map(to_tuple, poly_sol))` before applying those denominator exclusions. This is a new failure mode with no public precedent.

Artifact (GitHub). https://github.com/xinjiegit/agent-ic-bug-hunt-sympy/tree/main/released_bugs/bug-013-nonlinsolve-accepts-an-undefined-point-as-a-solution/

Case 14: Rayleigh CDF is positive below support

Task. Compute the CDF of a Rayleigh random variable below its support.

$$R \sim \text{Rayleigh}(1), \quad F_R(-1) = \text{cdf}(R)(-1)$$

SymPy's answer.

$$\text{cdf}(R)(-1) = 1 - e^{-1/2}.$$

Correct answer.

$$\text{cdf}(R)(-1) = 0.$$

Explanation. A Rayleigh random variable has support $[0, \infty)$, so the event $R \leq -1$ is empty. SymPy is extending the nonnegative-branch formula $1 - e^{-z^2/2}$ to $z < 0$, where the CDF should instead be zero.

Likely root cause. The diagnosis points to `sympy/stats/crv_types.py: RayleighDistribution._cdf` returns an unguarded formula even though the distribution declares support $[0, \infty)$, and `sympy/stats/crv.py` accepts that custom CDF before generic support handling. Deduplication frames this as a new failure mode with no public precedent.

Artifact (GitHub). https://github.com/xinjiegit/agent-ic-bug-hunt-sympy/tree/main/released_bugs/bug-014-rayleigh-cdf-is-positive-below-support/

Case 15: uniform quantile misses the $p = 1$ endpoint

Task. Compute the $p = 1$ quantile of a unit uniform random variable.

$$U \sim \text{Uniform}(0, 1), \quad Q(1) = \text{quantile}(U)(1).$$

SymPy's answer.

$$Q(1) = \emptyset.$$

Correct answer.

$$Q(1) = 1.$$

Explanation. The documented quantile definition is

$$Q(p) = \inf\{x \in \mathbb{R} : p \leq F(x)\},$$

where F is the cumulative distribution function. For $U \sim \text{Uniform}(0, 1)$, the CDF satisfies $F(x) < 1$ for $x < 1$ and $F(1) = 1$. Thus $Q(1) = 1$.

SymPy's quantile API reports quantiles as sets, so the expected API-level result is $\{1\}$; the alarming behavior is that it returns $\emptyset$, meaning that no value is reported for this endpoint quantile.

Likely root cause. The generic continuous quantile implementation in `sympy/stats/crv.py` solves $F(x) = p$ over the support; for $\text{Uniform}(0, 1)$ this produces an inverse intersected with `Interval.Ropen(0, 1)`, so substituting $p = 1$ drops the endpoint. Deduplication frames this as a new instance of a known failure mode.

Artifact (GitHub). https://github.com/xinjiegit/agent-ic-bug-hunt-sympy/tree/main/released_bugs/bug-015-uniform-quantile-misses-the-p-1-endpoint/

Case 16: incorrect Laplace transform of $\text{Heaviside}(a - t)$

Task. Compute the Laplace transform of a closing step with real cutoff a .

$$\mathcal{L}\{\text{Heaviside}(a - t)\}(s), \quad a \in \mathbb{R}, s > 0.$$

SymPy's answer for symbolic a .

$$\mathcal{L}_{\text{SymPy}}\{\text{Heaviside}(a - t)\}(s) = \frac{e^{-as}}{s}.$$

Correct answer.

$$\mathcal{L}\{\text{Heaviside}(a - t)\}(s) = \begin{cases} \frac{1 - e^{-as}}{s}, & a > 0, \\ 0, & a \leq 0. \end{cases}$$

Explanation. The Laplace integral runs over $[0, \infty)$: this step contributes on $[0, a)$ when $a > 0$, and on no interval when $a \leq 0$. That gives the piecewise result above, not e^{-as}/s .

Likely root cause. In `sympy/integrals/laplace.py`, the symbolic path returns e^{-as}/s guarded by the impossible condition $(a > 0) \wedge (a < 0) \wedge \text{Ne}(1/a, 0)$; `noconds=True` discards that guard.

Artifact (GitHub). https://github.com/xinjiegit/agent-ic-bug-hunt-sympy/tree/main/released_bugs/bug-016-incorrect-laplace-transform-of-heaviside-a-t/

Case 17: telescoping sum ignores undefined initial terms

Task. Evaluate the infinite sum

$$\sum_{k=0}^{\infty} \frac{1}{k(k-1)}.$$

SymPy's answer.

$$\text{Sum}\left(\frac{1}{k(k-1)}, (k, 0, \infty)\right). \text{doit()} = -1.$$

Correct answer.

$$\sum_{k=0}^{\infty} \frac{1}{k(k-1)} \text{ is undefined.}$$

Explanation. The terms at $k = 0$ and $k = 1$ have denominator zero, so although $\frac{1}{k(k-1)} = \frac{1}{k-1} - \frac{1}{k}$ telescopes on ranges where the terms are defined, the requested range begins with two undefined terms. The well-defined tail from $k = 2$ sums to 1, so the returned value -1 is not a valid value for the tail either.

Likely root cause. The diagnosis points to the telescoping path in `sympy/concrete/summations.py`: after partial fractions, SymPy applies the endpoint telescoping formula without checking for poles in the original summation range. This is a new instance of a known failure mode.

Artifact (GitHub). https://github.com/xinjiegit/agent-ic-bug-hunt-sympy/tree/main/released_bugs/bug-017-telescoping-sum-ignores-undefined-initial-terms/

Case 18: $\sinh$ incorrectly evaluates $I\pi + I(\pi - 1)$ to zero

Task. Evaluate a hyperbolic sine at a split imaginary-period argument.

$$\sinh(I\pi + I(\pi - 1))$$

SymPy's answer.

$$\sinh(I\pi + I(\pi - 1)) = 0.$$

Correct answer.

$$\sinh(I\pi + I(\pi - 1)) = -I \sin(1).$$

Explanation. The argument satisfies $I\pi + I(\pi - 1) = I(2\pi - 1)$, and $\sinh(It) = I \sin(t)$. Hence the value is $I \sin(2\pi - 1) = -I \sin(1)$, not zero.

Likely root cause. The diagnosis identifies `_peeloff_pi` in `sympy/functions/elementary/trigonometric.py` as peeling the π -coefficient from the nested addend $\pi - 1$ while dropping the non- π remainder. The deduplication assessment classifies this as a new instance of a known failure mode.

Artifact (GitHub). https://github.com/xinjiegit/agent-ic-bug-hunt-sympy/tree/main/released_bugs/bug-018-sinh-incorrectly-evaluates-i-pi-i-pi-1-to-zero/

Case 19: recurrence solver violates initial conditions

Task. Solve the recurrence with the supplied initial conditions.

$$y(n+2) - (n+2)y(n+1) + y(n) = 0, \quad y(0) = y(1) = 1.$$

SymPy's answer. SymPy's `rsolve` returns the zero sequence as the solution.

Correct answer. A valid answer must satisfy $y(0) = 1$ and $y(1) = 1$, or the solver should report that it cannot produce such a solution; the zero sequence is not a correct answer.

Explanation. Substituting into SymPy's returned expression gives $0|_{n=0} = 0$ and $0|_{n=1} = 0$, contradicting both supplied initial conditions. Forward recurrence from the requested values starts $1, 1, 1, 2, 7, 33, \dots$, so this is not an equivalent representation of the requested initial-value solution.

Likely root cause. The diagnosis locates the issue in `sympy/solvers/recr.py:rsolve`: initial conditions are applied only when the internal solution contains arbitrary constants. Here `rsolve_hyper` returns the constant-free candidate 0 with no symbols, so validation is skipped; this is a new instance of a known failure mode.

Artifact (GitHub). https://github.com/xinjiegit/agent-ic-bug-hunt-sympy/tree/main/released_bugs/bug-019-recurrence-solver-violates-initial-conditions/

Case 20: conditional uniform density keeps old support

Task. Compute the density of a uniform random variable conditioned on being larger than $1/2$.

$$Y = \text{given}(U, U > 1/2), \quad U \sim \text{Uniform}(0, 1).$$

SymPy’s answer.

$$\text{density}(Y)(x) = 2 \cdot \mathbf{1}_{[0,1]}(x).$$

Correct answer.

$$\text{density}(Y)(x) = 2 \cdot \mathbf{1}_{(1/2,1]}(x).$$

Explanation. Conditioning on $U > 1/2$ must both divide by $P(U > 1/2) = 1/2$ and restrict the support to $(1/2, 1]$. Note that SymPy’s returned density has total mass 2.

Likely root cause. The diagnosis identifies `ContinuousPSPACE.conditional_space` in `sympy/stats/crv.py` as storing only the renormalized original PDF, while `compute_density` returns it unchanged when no variables are marginalized. This is a new failure mode with no public precedent.

Artifact (GitHub). https://github.com/xinjiegit/agent-ic-bug-hunt-sympy/tree/main/released_bugs/bug-020-conditional-uniform-density-keeps-old-support/

4 Conclusion

Across our runs, the harness found many silent SymPy bugs in routine algebra and calculus—precisely the computations users expect a computer algebra system to handle reliably, especially one as widely used as SymPy. SymPy is a core piece of mathematical infrastructure and underpins downstream systems such as SageMath [1, 2]. It is also increasingly used as a backend for LLM-based mathematical reasoning systems [11, 15, 16].

Our work fits into an emerging model for software maintenance. LLM agents continuously probe existing software, turn failures into reproducible examples, and draft candidate pull requests, while human maintainers decide which failures are real bugs, what the correct behavior should be, and which patches to merge. Through this kind of iteration, we hope to bring SymPy’s reliability closer to the ideal perfection.

This report has several limitations. It is a curated case study, and is not meant to be a comprehensive report of all mathematical errors of SymPy. In particular, it does not measure how many bugs SymPy contains, and the absence of a discovered bug in a subsystem should not be construed as evidence of correctness. The harness is also stochastic, so different model versions, prompts, time budgets, target commits, or package versions may produce different candidates. Finally, deduplication and source-level diagnosis are both best-effort: a finding may be a new instance of a broader known failure mode, and the diagnosis should be read as engineering evidence rather than a definitive patch plan.

References

- [1] A. Meurer, C. P. Smith, M. Paprocki, O. Čertík, S. B. Kirpichev, M. Rocklin, A. Kumar, S. Ivanov, J. K. Moore, S. Singh, T. Rathnayake, S. Vig, B. E. Granger, R. P. Muller, F. Bonazzi, H. Gupta, S. Vats, F. Johansson, F. Pedregosa, M. J. Curry, A. R. Terrel, Š. Roučka, A. Saboo, I. Fernando, S. Kulal, R. Cimrman, and A. Scopatz. SymPy: Symbolic Computing in Python. *PeerJ Computer Science*, 3:e103, 2017.
- [2] The Sage Developers. *SageMath, the Sage Mathematics Software System*. <https://www.sagemath.org>.
- [3] G. Gede, D. L. Peterson, A. S. Nanjangud, J. K. Moore, and M. Hubbard. Constrained Multibody Dynamics with Python: From Symbolic Equation Generation to Publication. In *Proc. ASME IDETC/CIE*, 2013. DOI:10.1115/DETC2013-13470.
- [4] R. Cimrman. SfePy — Write Your Own FE Application. In *Proc. 6th European Conference on Python in Science (EuroSciPy 2013)*, 2014. arXiv:1404.6391.
- [5] M. J. Wester. A Critique of the Mathematical Abilities of CA Systems. In M. J. Wester, editor, *Computer Algebra Systems: A Practical Guide*, pages 25–60. Wiley, 1999.
- [6] E. T. Barr, M. Harman, P. McMinn, M. Shahbaz, and S. Yoo. The Oracle Problem in Software Testing: A Survey. *IEEE Transactions on Software Engineering*, 41(5):507–525, 2015.
- [7] T. Y. Chen, F.-C. Kuo, H. Liu, P.-L. Poon, D. Towey, T. H. Tse, and Z. Q. Zhou. Metamorphic Testing: A Review of Challenges and Opportunities. *ACM Computing Surveys*, 51(1):4:1–4:27, 2018.
- [8] Y. Deng, C. S. Xia, H. Peng, C. Yang, and L. Zhang. Large Language Models Are Zero-Shot Fuzzers: Fuzzing Deep-Learning Libraries via Large Language Models. In *Proc. ISSTA*, 2023. arXiv:2212.14834.

- [9] C. S. Xia, M. Paltenghi, J. L. Tian, M. Pradel, and L. Zhang. Fuzz4All: Universal Fuzzing with Large Language Models. In *Proc. ICSE*, 2024. arXiv:2308.04748.
- [10] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? In *Proc. ICLR*, 2024. arXiv:2310.06770.
- [11] L. Gao, A. Madaan, S. Zhou, U. Alon, P. Liu, Y. Yang, J. Callan, and G. Neubig. PAL: Program-aided Language Models. In *Proc. ICML*, 2023. arXiv:2211.10435.
- [12] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao. ReAct: Synergizing Reasoning and Acting in Language Models. In *Proc. ICLR*, 2023. arXiv:2210.03629.
- [13] OpenAI. Codex: AI coding partner from OpenAI. <https://openai.com/codex/>, 2025. Accessed June 2026.
- [14] OpenAI. GPT-5.6: Frontier intelligence that scales with your ambition. <https://openai.com/index/gpt-5-6/>, 2026. Accessed July 2026.
- [15] J. He-Yueya, G. Poesia, R. E. Wang, and N. D. Goodman. Solving Math Word Problems by Combining Language Models With Symbolic Solvers. In *The 3rd Workshop on Mathematical Reasoning and AI (MATH-AI), NeurIPS*, 2023. arXiv:2304.09102.
- [16] S. Bagheri Nezhad, Y. Li, and A. Agrawal. SymCode: A Neurosymbolic Approach to Mathematical Reasoning via Verifiable Code Generation. In *Findings of the ACL: EACL*, pages 1489–1503, 2026. arXiv:2510.25975.
- [17] Anthropic. Claude Code: Anthropic’s agentic coding system. <https://claude.com/product/claude-code>, 2025. Accessed June 2026.

A Technical report

A.1 Scope

The catalog is curated across multiple harness runs as a set of strong, individually reproducible witnesses, confirmed against SymPy 1.14.0 and the most recent development branch as of publishing this report, at commit `15e21aca70`. Each is a silent correctness failure—a definite, well-formed answer that is mathematically wrong.

A.2 Setup

The harness (`scripts/run_harness.py`) drives six agents, each a separate non-interactive `codex exec` invocation of OpenAI’s Codex CLI: the bug-hunting, verification, root-finding, deduplication, artifact, and write-up agents. All six run the same model, GPT-5.6 Sol (medium reasoning effort), with `--sandbox workspace-write` and `--ask-for-approval never`—fully autonomously, with no human approval in the loop; only the deduplication agent additionally enables web search (`--search`) to look for public duplicates of a candidate. We drive the Codex CLI entirely through a ChatGPT subscription rather than an API key, so a complete run—candidate search, independent verification, source-level diagnosis, deduplication, and artifact packaging—incur no metered API usage and can be reproduced on a workstation that holds a SymPy checkout. Each run is bounded by configurable stopping limits (by default at most 10 confirmed bugs, a 4-hour wall-clock budget, and 3 consecutive empty discovery passes).

A.3 Catalog composition

Catalog composition	Count
Confirmed bugs with artifacts	20
New instance of a known failure mode	16
New failure mode (no public precedent)	4

Table 2: Composition of the curated catalog.

A.4 Detailed Per-Bug Reports

A.4.1 Bug 1: `solveset` returns 0 for `asin` outside the principal range

Minimal Reproducer

```

import sympy
from sympy import Eq, S, asin, pi, simplify, solveset, symbols

x = symbols("x")
eq = Eq(asin(x), pi)
sol = solveset(eq, x, domain=S.Complexes)

print("SymPy version:", sympy.__version__)
print("solveset result:", sol)
print("residual at returned value 0:",
      simplify(eq.lhs.subs(x, 0) - eq.rhs.subs(x, 0)))

```

Observed Behavior

```

SymPy version: 1.14.0
solveset result: {0}
residual at returned value 0: -pi

```

SymPy returns the finite solution set $\{0\}$. However, direct substitution of the returned value into the original equation gives

$$\operatorname{asin}(0) - \pi = -\pi \neq 0.$$

Thus the returned element does not satisfy the equation that `solveset` was asked to solve.

Expected Behavior

The mathematically correct result is $\emptyset$, represented in SymPy as `EmptySet`. There is no complex value x for which SymPy's principal inverse sine satisfies $\operatorname{asin}(x) = \pi$.

Mathematical Explanation

SymPy's `asin` is the principal inverse sine. On real inputs its image is $[-\pi/2, \pi/2]$, and more generally the complex principal branch has a fixed principal image rather than all complex values. The target π is outside that principal image.

Applying `sin` to both sides gives $x = \sin(\pi) = 0$, but this implication is not reversible for the principal inverse sine. The value $x = 0$ satisfies $\sin(\pi) = 0$, but it does not satisfy the original equation because $\operatorname{asin}(0) = 0$, not π . The wrong answer is therefore not a harmless branch choice, a missing constant, or an equivalent representation of a solution; it is a concrete non-solution.

Source-Level Diagnosis

The diagnosis identifies the relevant code path in `sympy/solvers/solveset.py`, in the complex inversion helper `_invert_complex` near line 550. The traced

call path is that `solveset` converts `Eq(asin(x), pi)` to `asin(x) - pi = 0` and enters `_solveset`. The expression is not handled by `_solve_trig`, because the top-level function is the inverse trigonometric function `asin`, not `sin`. The solver then calls `_invert(..., domain=S.Complexes)`, which dispatches to `_invert_complex`.

For this equation, `_invert_complex(asin(x) - pi, {0}, x)` removes the additive constant and then handles `asin(x)` through the generic function-inverse path. It applies `asin.inverse()`, i.e. `sin`, to the right-hand side and produces $(x, \{\sin(\pi)\}) = (x, \{0\})$. The problematic source rule is the generic use of a function's inverse when one is available:

```
return _invert_complex(f.args[0],
                      imageset(Lambda(n, f.inverse()(n)), g_ys), symbol)
```

For principal `asin`, this treats `sin` as a globally valid inverse without carrying the required range condition on the right-hand side. The later filtering accepts the finite inversion result after `domain_check` rather than checking the residual $\text{asin}(0) - \pi$ against the original equation. Consequently, the candidate 0 is accepted even though it is not a solution. A plausible fix direction supported by the diagnosis is to make inverse-function handling for principal inverse functions carry image or range conditions, returning $\emptyset$ or a `ConditionSet` when a concrete target such as π violates the principal `asin` image.

Additional Failing Instances

The same failure occurs for integer multiples of π that lie outside the principal image of `asin`. The independent verification checks

$$\text{asin}(x) = n\pi, \quad n \in \{1, 2, -1, 3, 4, 5\},$$

over the complex numbers. In each case, SymPy's inversion computes $x = \sin(n\pi) = 0$, but the returned value has residual $\text{asin}(0) - n\pi = -n\pi \neq 0$. Since none of these targets belongs to the principal image of `asin`, the expected solution set is $\emptyset$ in all cases.

Input / Expression	SymPy behavior	Expected behavior
<code>solveset(asin(x) = π, x, $\mathbb{C}$)</code>	returns $\{0\}$; residual $-\pi$	$\emptyset$
<code>solveset(asin(x) = 2π, x, $\mathbb{C}$)</code>	returns $\{0\}$; residual -2π	$\emptyset$
<code>solveset(asin(x) = $-\pi$, x, $\mathbb{C}$)</code>	returns $\{0\}$; residual π	$\emptyset$
<code>solveset(asin(x) = 3π, x, $\mathbb{C}$)</code>	returns $\{0\}$; residual -3π	$\emptyset$
<code>solveset(asin(x) = 4π, x, $\mathbb{C}$)</code>	returns $\{0\}$; residual -4π	$\emptyset$
<code>solveset(asin(x) = 5π, x, $\mathbb{C}$)</code>	returns $\{0\}$; residual -5π	$\emptyset$

Related Issues Assessment

The bug-finding harness performed a best-effort deduplication check. The closest related public material found was documentation confirming the principal-branch behavior of `asin`, general principal-branch references, and SymPy documentation stating that `solveset` returns values for which the equation is true. No public issue, pull request, discussion, or external report was identified for `asin( $x$ ) =  $\pi$` , the wrong output $\{0\}$, or a missing principal-range check in `solveset` for this exact case. The deduplication verdict is therefore a new instance of a known failure mode: the general need for branch- and range-aware inverse solving is known, but this specific reproducible case appears previously unreported and remains individually actionable as an issue and regression test.

Proposed SymPy Regression Test

```
import pytest

from sympy import Eq, FiniteSet, S, asin, pi, simplify, solveset, symbols

@pytest.mark.parametrize("n", [1, 2, -1, 3, 4, 5])
def test_solveset_principal_asin_rejects_out_of_range_targets(n):
    x = symbols("x")
    eq = Eq(asin(x), n*pi)
    sol = solveset(eq, x, domain=S.Complexes)

    assert sol != FiniteSet(0)
    assert simplify(eq.lhs.subs(x, 0) - eq.rhs.subs(x, 0)) != 0
```

A.4.2 Bug 2: solveset Returns All Complexes for $\sqrt{x^2} = x$

Minimal Reproducer

```
from sympy import Eq, N, S, simplify, solveset, sqrt, symbols

x = symbols("x")
eq = Eq(sqrt(x**2), x)
sol = solveset(eq, x, domain=S.Complexes)

print("solveset result:", sol)
print("residual at x = -1:", simplify(eq.lhs.subs(x, -1) - eq.rhs.subs(x, -1)))
print("numeric residual:", N(eq.lhs.subs(x, -1) - eq.rhs.subs(x, -1), 50)
)
```

Observed Behavior

```
solveset result: Complexes
residual at x = -1: 2
numeric residual: 2.0000000000000000000000000000000000000000000000000000000
```

SymPy reports that every complex number solves

$$\sqrt{x^2} = x.$$

This is incorrect because the returned set includes $x = -1$, and direct substitution gives

$$\sqrt{(-1)^2} - (-1) = 1 + 1 = 2.$$

Expected Behavior

The result must be a proper subset of $\mathbb{C}$ and must exclude $x = -1$. With the principal square root, the complex solutions are the branch-restricted set

$$\{z \in \mathbb{C} : \operatorname{Re}(z) > 0\} \cup \{it : t \in \mathbb{R}, t \geq 0\}.$$

In particular, over the real line the solution set is $[0, \infty)$, not all real or complex values.

Mathematical Explanation

The identity $\sqrt{x^2} = x$ is not valid for arbitrary complex x . SymPy's $\sqrt{\cdot}$ denotes the principal square root, so clearing the square root by squaring both sides can introduce extraneous solutions unless the principal-branch condition is preserved.

For real x , the familiar identity is

$$\sqrt{x^2} = |x|,$$

so the equation $\sqrt{x^2} = x$ holds exactly for $x \geq 0$. The same issue persists over the complex numbers: the principal square root chooses a branch, and $\sqrt{z^2}$ agrees with z only on the corresponding branch-restricted subset. The returned answer $\mathbb{C}$ is therefore not a harmless alternative representation or a missing simplification; it contains concrete values that do not satisfy the original equation.

Source-Level Diagnosis

The diagnosis identifies the immediate source-level cause in `sympy/solvers/solveset.py`, specifically in `_solve_radical` around line 1094. The traced call path is:

```
solveset → _solveset → unrad → _solve_radical.
```

For $\sqrt{x^2} - x = 0$, direct inversion does not reduce the equation, so `_solveset` calls `unrad`. The radical-removal step returns the unradicalized equation $0 = 0$ with no covering variables. In other words, squaring has turned the original branch-sensitive equation into a tautology.

The problematic behavior is that `_solve_radical` treats the solution set of this unradicalized equation as acceptable even when it is non-finite. The diagnosis notes that `unrad` produces a superset equation rather than an equivalent equation, but the checker in `_solve_radical` validates only certain finite or set-combination cases. Non-finite set results such as $\mathbb{C}$ can therefore be returned unchanged, leaving no check against the original equation.

```
if not cov:
    result = solveset_solver(eq, symbol) - ...
...
else:
    return solutions
```

This behavior causes the observed failure because the branch condition lost during radical clearing is never restored. A plausible fix direction identified by the diagnosis is for `_solve_radical` not to return non-finite radical-clearing results unconditionally. It should either preserve the original equation as a condition, for example through a `ConditionSet`, or derive and apply the branch conditions introduced by radical removal.

Additional Failing Instances

The independent verification covers the representative case and shifted variants

$$\sqrt{(x+a)^2} = x+a$$

for $a \in \{0, 1, 2, 3, 4, 5\}$. In each case, SymPy returns $\mathbb{C}$, while the explicit counterexample $x = -a - 1$ makes $x + a = -1$. Therefore the residual is again

$$\sqrt{(-1)^2} - (-1) = 2,$$

so the same principal-square-root branch error appears across the family.

Input / Expression	SymPy behavior	Expected behavior
<code>solveset($\sqrt{x^2} = x, x, \mathbb{C}$)</code>	Returns $\mathbb{C}$; includes $x = -1$.	A proper subset of $\mathbb{C}$; exclude $x = -1$.
<code>solveset($\sqrt{(x+1)^2} = x+1, x, \mathbb{C}$)</code>	Returns $\mathbb{C}$; includes $x = -2$.	A proper subset of $\mathbb{C}$; exclude $x = -2$.
<code>solveset($\sqrt{(x+2)^2} = x+2, x, \mathbb{C}$)</code>	Returns $\mathbb{C}$; includes $x = -3$.	A proper subset of $\mathbb{C}$; exclude $x = -3$.
<code>solveset($\sqrt{(x+3)^2} = x+3, x, \mathbb{C}$)</code>	Returns $\mathbb{C}$; includes $x = -4$.	A proper subset of $\mathbb{C}$; exclude $x = -4$.
<code>solveset($\sqrt{(x+4)^2} = x+4, x, \mathbb{C}$)</code>	Returns $\mathbb{C}$; includes $x = -5$.	A proper subset of $\mathbb{C}$; exclude $x = -5$.
<code>solveset($\sqrt{(x+5)^2} = x+5, x, \mathbb{C}$)</code>	Returns $\mathbb{C}$; includes $x = -6$.	A proper subset of $\mathbb{C}$; exclude $x = -6$.

Related Issues Assessment

The bug-finding harness performed a best-effort deduplication check and classified this as a new instance of a known failure mode. The closest public material is SymPy’s own square-root documentation, which explicitly explains that $\sqrt{x^2}$ does not simplify to x in general and even uses $x = -1$ as a counterexample, together with general `solveset` documentation stating that returned sets should contain solutions of the input equation. Those references establish that the mathematical pitfall is known, but they do not report the specific solver behavior `solveset( $\sqrt{x^2} = x, x, \mathbb{C}$ ) =  $\mathbb{C}$` . The case remains individually actionable because it is a reproducible solver correctness failure with a concise regression test.

Proposed SymPy Regression Test

```

import pytest

from sympy import Eq, S, simplify, solveset, sqrt, symbols

@pytest.mark.parametrize("a", [0, 1, 2, 3, 4, 5])
def test_solveset_sqrt_square_not_all_complexes(a):
    x = symbols("x")
    eq = Eq(sqrt((x + a)**2), x + a)
    sol = solveset(eq, x, domain=S.Complexes)
    bad = -a - 1

    assert sol != S.Complexes
    assert simplify(eq.lhs.subs(x, bad) - eq.rhs.subs(x, bad)) != 0

```

A.4.3 Bug 3: solveset returns nan as a zero of secant

Minimal Reproducer

```

from sympy import S, cos, sec, solveset, symbols

x = symbols("x")

solution = solveset(sec(x), x, domain=S.Complexes)
print("solveset(sec(x), x, domain=S.Complexes) =", solution)
print("sec(nan) =", sec(S.NaN))
print("solveset(1/cos(x), x, domain=S.Complexes) =",
      solveset(1 / cos(x), x, domain=S.Complexes))

```

Observed Behavior

```

SymPy version: 1.14.0
solveset(sec(x), x, domain=S.Complexes) = {nan}
sec(nan) = nan
solveset(1/cos(x), x, domain=S.Complexes) = EmptySet

```

SymPy returns a singleton set containing `nan` as a zero of $\sec(x)$. Direct evaluation of the reported candidate gives $\sec(\text{nan}) = \text{nan}$, not 0, and the mathematically equivalent reciprocal spelling $1/\cos(x)$ returns $\emptyset$.

Expected Behavior

The expected result is

$$\text{solveset}(\sec(x), x, \mathbb{C}) = \emptyset.$$

The value `nan` is not a complex number and should never be returned as a solution of the original equation.

Mathematical Explanation

For every finite complex number z , $\cos(z)$ is finite. Therefore

$$\sec(z) = \frac{1}{\cos(z)}$$

can never equal 0. If $\cos(z) \neq 0$, then $\sec(z)$ is a nonzero finite reciprocal. If $\cos(z) = 0$, then $\sec(z)$ has a pole and is undefined or infinite, not zero. Thus there is no finite complex z satisfying

$$\sec(z) = 0.$$

The returned value is not a harmless alternative representation or a branch choice: `nan` is not an element of the complex solution domain, and substitution does not satisfy the equation.

Source-Level Diagnosis

The diagnosis identifies the relevant source file as `sympy/solvers/solveset.py`. The public call `solveset(sec(x), x, domain=S.Complexes)` reaches `_solveset`; the fallback inversion path then calls `invert_complex(f, 0, symbol, domain)` around lines 2321–2324. `invert_complex` delegates to `_invert_complex`, which recognizes `sec(x)` as a trigonometric function around lines 577–578 and then calls `_invert_trig_hyp_complex`.

The problematic rule is in `_invert_trig_hyp_complex` around lines 462–466: the inversion helper treats `sec` like an ordinary invertible trigonometric function, chooses `asec` for the inverse, and constructs families of the form $2n\pi + \operatorname{asec}(a)$ and $2n\pi - \operatorname{asec}(a)$. For the equation $\sec(x) = 0$, this applies `asec(0)`. In SymPy 1.14.0, `asec(0)` evaluates to `zoo`; arithmetic such as $2n\pi \pm \operatorname{zoo}$ simplifies during set construction to `{nan}`. The inversion code does not first enforce the reciprocal-trig range condition that `sec` and `csc` cannot take the value 0, and it does not later filter non-finite or `nan` candidates from the returned finite set.

```
# Simplified from the diagnosed inversion step:
# sec(x) = 0 is inverted as
x = 2*n*pi + asec(0)
x = 2*n*pi - asec(0)
```

This source-level behavior turns an impossible equation into invalid inverse images. A plausible fix direction supported by the diagnosis is to add range and singularity guards in the trigonometric inversion helper: for reciprocal trigonometric functions such as `sec` and `csc`, exclude 0 from the target values before applying `asec` or `acsc`; if no target values remain, return `S.EmptySet`. A defensive cleanup could also discard `nan`, ∞ , $-\infty$, and complex infinity from finite solution sets.

Additional Failing Instances

The independent verification checks the representative case and five additional instances of the same form,

$$a \sec(bx + c),$$

where $a \neq 0$. Each expression has no complex zeros for the same reason: away from poles it is a nonzero scalar times a reciprocal of a finite cosine, and at poles it is undefined or infinite rather than zero. SymPy nevertheless returns `{nan}` for each listed case, while the equivalent reciprocal spelling $a/\cos(bx + c)$ returns $\emptyset$.

Input / Expression	SymPy behavior	Expected behavior
$\sec(x)$	Returns <code>{nan}</code>	$\emptyset$
$\sec(x + 1)$	Returns <code>{nan}</code>	$\emptyset$
$\sec(x - 2)$	Returns <code>{nan}</code>	$\emptyset$
$\sec(2x)$	Returns <code>{nan}</code>	$\emptyset$
$\sec(3x + 1)$	Returns <code>{nan}</code>	$\emptyset$
$2 \sec(x)$	Returns <code>{nan}</code>	$\emptyset$

Related Issues Assessment

The bug-finding harness performed a best-effort deduplication check and classified this as a new instance of a known failure mode. The closest related public material is SymPy's solveset documentation, SymPy's elementary-function documentation for reciprocal trigonometric functions, and the SymPy 1.14.0 source code showing the sec/csc inversion path through `asec/acsc`. The artifact did not identify a public issue, pull request, release note, Stack Overflow report, or discussion describing this exact return value for `solveset(sec(x), x, domain=S.Complexes)`. The general reciprocal-trig inversion failure mode may be broadly understood, but this exact reproducible case is previously unreported and remains individually actionable as an issue and regression test.

Proposed SymPy Regression Test

```
import pytest

from sympy import S, sec, solveset, symbols

@pytest.mark.parametrize(
    "scale, slope, offset",
    [
        (1, 1, 0),
```

```

        (1, 1, 1),
        (1, 1, -2),
        (1, 2, 0),
        (1, 3, 1),
        (2, 1, 0),
    ],
)
def test_solveset_sec_has_no_complex_zero(scale, slope, offset):
    x = symbols("x")
    assert solveset(scale * sec(slope * x + offset), x, domain=S.
Complexes) == S.EmptySet

```

A.4.4 Bug 4: Integral of $\log(x)/(x^2 - 1)$ Differentiates Back with an Extra Imaginary Term

Minimal Reproducer

```

import os
import sys

SYMPY_CHECKOUT_PATH = os.environ.get("SYMPY_CHECKOUT_PATH")
if SYMPY_CHECKOUT_PATH:
    sys.path.insert(0, SYMPY_CHECKOUT_PATH)

import sympy
from sympy import * # noqa: F401,F403

print("SymPy version:", sympy.__version__)
print("SymPy file:", sympy.__file__)
print("Python executable:", sys.executable)
if SYMPY_CHECKOUT_PATH and not os.path.abspath(sympy.__file__).startswith(
    os.path.abspath(SYMPY_CHECKOUT_PATH)):
    raise RuntimeError("SymPy was not imported from SYMPY_CHECKOUT_PATH")

x = symbols("x")
f = log(x)/(x**2 - 1)
F = integrate(f, x)
residual = simplify(diff(F, x) - f)
print("integrate(log(x)/(x**2 - 1), x) =", F)
print("diff(integral, x) - log(x)/(x**2 - 1) =", residual)
print("residual at x=2 =", N(residual.subs(x, 2), 50))

```

Observed Behavior

```

SymPy version: 1.14.0
SymPy file: $SYMPY_CHECKOUT_PATH/sympy/__init__.py

```

```
Python executable: python3
residual at x=2 = 0.e-56 -
1.5707963267948966192313216916397514420985846996876*I
```

The returned antiderivative differentiates to the original real integrand plus a nonzero imaginary residual at the ordinary positive real point $x = 2$. The residual is $-1.57079632679\dots i = -\pi i/2$, so this is not a formatting difference, an unevaluated zero, or a harmless alternative constant of integration.

Expected Behavior

For

$$f(x) = \frac{\log x}{x^2 - 1},$$

any valid local antiderivative F on a neighborhood of $x = 2$ should satisfy

$$F'(2) = f(2) = \frac{\log 2}{3}.$$

Equivalently, $(F' - f)|_{x=2}$ should simplify or numerically evaluate to 0.

Mathematical Explanation

The integrand

$$f(x) = \frac{\log x}{x^2 - 1}$$

is real-valued and analytic near $x = 2$, since $x = 2$ is away from the poles at $x = \pm 1$ and away from the principal logarithm branch point. The defining local property of an indefinite integral is therefore unambiguous: on such a neighborhood, $F'(x) = f(x)$.

SymPy's returned expression instead gives

$$F'(2) - f(2) = -\frac{\pi i}{2}.$$

An additive constant in an antiderivative would disappear under differentiation, so the residual cannot be explained by a different integration constant. It is also not merely a different branch-equivalent representation at the tested point: the derivative itself contains an extra branch term. Since $f(2) = \log(2)/3$ is real, the nonzero imaginary residual makes the returned expression fail the direct antiderivative check.

Source-Level Diagnosis

The diagnosis locates the relevant code in `sympy/integrals/meijerint.py`, specifically the shifted-logarithm lookup rule around line 196 and the Meijer indefinite integration functions `meijerint_indefinite` and `_meijerint_`.

`indefinite_1` around lines 1653–1740. The public call starts with the original integration request and enters `Integral._eval_integral`. Direct Risch and heurisch paths do not solve the original integrand. Then `manualintegrate` applies integration by parts, reducing the problem to logarithmic subintegrals:

```
integrate(log(x)/(x**2 - 1), x)

(log(x - 1)/2 - log(x + 1)/2)*log(x)
- Integral(log(x - 1)/x, x)/2
+ Integral(log(x + 1)/x, x)/2
```

Because this expression still contains unevaluated `Integral` terms, `Integral._eval_integral` recursively evaluates those subintegrals. The diagnosis identifies `Integral(log(x - 1)/x, x)` as the faulty subproblem: it is solved by the Meijer integration path after rewriting $\log(x - 1)$ through the table entry for shifted logarithms. For this shifted logarithm, `_rewrite_single` returns a branch-sensitive representation involving $x \exp_{\text{polar}}(-i\pi)$ and explicit $i\pi$ constants.

The resulting sub-antiderivative differentiates on the positive side of the branch point to

$$\frac{\log(x - 1)}{x} + \frac{i\pi}{x},$$

rather than to $\log(x - 1)/x$. In the full integration-by-parts result, this subintegral appears with coefficient $-1/2$, so the extra derivative term contributes an incorrect branch residual. At $x = 2$, the observed residual is $-\pi i/2$. The diagnosis therefore points to a branch-sensitive Meijer/polylogarithm expression being accepted as an antiderivative for the principal-log integrand on a real interval where it differentiates to the wrong branch.

A plausible fix direction supported by the diagnosis is for the Meijer rewrite for shifted logarithms either to preserve the principal branch of $\log(x - 1)$ under the shift/sign rewrite or to decline this transformation when the shift crosses the logarithm branch cut. A conservative implementation could leave `meijerint_indefinite(log(x - 1)/x, x)` unevaluated unless it can attach correct branch conditions.

Additional Failing Instances

The independent verification covers the family

$$f_a(x) = \frac{\log x}{x^2 - a^2}, \quad a \in \{1, 2, 3, 4, 5, 6\},$$

with the derivative residual checked at the regular positive real point $x = 2a$. The expected behavior is the same in every case: f_a is finite and analytic at $x = 2a$, and a valid local antiderivative must differentiate back to f_a , giving residual 0.

Input / Expression	SymPy behavior	Expected behavior
$\int \log(x)/(x^2 - 1) dx$, checked at $x = 2$	residual = $-\pi i/2$	residual = 0
$\int \log(x)/(x^2 - 4) dx$, checked at $x = 4$	residual = $-\pi i/4$	residual = 0
$\int \log(x)/(x^2 - 9) dx$, checked at $x = 6$	residual = $-\pi i/9$	residual = 0
$\int \log(x)/(x^2 - 16) dx$, checked at $x = 8$	residual = $-\pi i/16$	residual = 0
$\int \log(x)/(x^2 - 25) dx$, checked at $x = 10$	residual = $-\pi i/25$	residual = 0
$\int \log(x)/(x^2 - 36) dx$, checked at $x = 12$	residual = $-\pi i/36$	residual = 0

These cases show that the failure is not isolated to a single denominator or evaluation point. Across this quadratic family, the integration procedure introduces a branch contribution from shifted logarithmic subintegrals, while the mathematical antiderivative contract requires the residual to vanish at each tested regular real point.

Related Issues Assessment

The bug-finding harness performed a best-effort deduplication check. It found broad public material on logarithmic branch cuts, polylogarithmic expressions, Meijer-G integration, and SymPy’s indefinite integration contract. A focused re-check identified the open issue [sympy/sympy#13850](#) (“Integration result appears to use a different branch of polylog”), which reports the same shifted-log/polylog branch-cut antiderivative family for $\int \log(x)/(1+x) dx$; the present bug reduces by parts to exactly the $\int \log(x \pm 1)/x dx$ subintegrals, so it is the same known family (see also [sympy/sympy#23337](#) for a related spurious $i\pi$ integration artifact). No prior report contained the exact expression $\log(x)/(x^2-1)$, the check at $x = 2$, the residual $-\pi i/2$, or the $\log(x)/(x^2 - a^2)$ family, so this specific instance is newly reported. The deduplication verdict is therefore a known family with a new specific instance, and this exact reproducible case remains individually actionable as an issue and regression test.

Proposed SymPy Regression Test

```
import pytest
from sympy import N, diff, integrate, log, simplify, symbols

@pytest.mark.parametrize("a", [1, 2, 3, 4, 5, 6])
```

```
def test_log_over_quadratic_antiderivative_differentiates_back(a):
    x = symbols("x")
    f = log(x)/(x**2 - a**2)
    F = integrate(f, x)
    residual = simplify(diff(F, x) - f)
    assert abs(complex(N(residual.subs(x, 2*a), 40))) < 1e-30
```

A.4.5 Bug 5: Wrong Negative-Real Branch in an Antiderivative

Minimal Reproducer

```
from sympy import N, diff, integrate, simplify, sqrt, symbols

x = symbols("x")
integrand = sqrt(x**2 + 1)/x
F = integrate(integrand, x)
residual = diff(F, x) - integrand

print("antiderivative:", F)
print("integrand at -2:", integrand.subs(x, -2))
print("derivative at -2:", diff(F, x).subs(x, -2))
print("residual at -2:", simplify(residual.subs(x, -2)))
print("numeric residual:", N(residual.subs(x, -2), 50))
```

Observed Behavior

```
antiderivative: x/sqrt(1 + x**(-2)) - asinh(1/x) + 1/(x*sqrt(1 + x**(-2)))
integrand at -2: -sqrt(5)/2
derivative at -2: sqrt(5)/2
residual at -2: sqrt(5)
numeric residual: 2.2360679774997896964091736687312762354406183596115
```

SymPy returns an expression as an antiderivative of

$$\frac{\sqrt{x^2 + 1}}{x}.$$

However, differentiating that returned expression and evaluating at $x = -2$ gives $\sqrt{5}/2$, while the original integrand evaluates to $-\sqrt{5}/2$. The residual is therefore $\sqrt{5}$, so this is not a harmless formatting difference or an omitted additive constant.

Expected Behavior

The derivative of an indefinite integral result should equal the integrand on each

interval where the integrand is defined. In particular, for

$$f(x) = \frac{\sqrt{x^2 + 1}}{x},$$

any antiderivative returned for the real variable x must differentiate back to $f(x)$ on both $(-\infty, 0)$ and $(0, \infty)$, up to the usual additive constants on those intervals. At $x = -2$, the derivative should be $-\sqrt{5}/2$, not $+\sqrt{5}/2$.

Mathematical Explanation

The sign error comes from replacing a branch-sensitive expression by its positive-real branch. For real nonzero x ,

$$\frac{\sqrt{x^2 + 1}}{x} = \text{sign}(x) \sqrt{1 + x^{-2}}.$$

Thus

$$\frac{\sqrt{x^2 + 1}}{x} = \sqrt{1 + x^{-2}} \quad \text{only when } x > 0,$$

whereas on the negative real axis it equals

$$-\sqrt{1 + x^{-2}}.$$

SymPy's returned expression differentiates to the positive branch $\sqrt{1 + x^{-2}}$. That derivative agrees with the integrand for positive real x , but has the wrong sign for negative real x . Since an additive constant disappears under differentiation, no choice of integration constant can repair a pointwise derivative residual of $\sqrt{5}$ at $x = -2$.

Source-Level Diagnosis

The diagnosis identifies the relevant integration path as the Meijer-G indefinite integration fallback. The public call `integrate( $\sqrt{x^2 + 1}/x, x$ )` reaches `Integral.doit` and `Integral._eval_integral` in `sympy/integrals/integrals.py`. The Risch, heuristic Risch, and manual integration paths do not produce an answer. The fallback calls `meijerint_indefinite` near `sympy/integrals/integrals.py:1105`. The result is then produced in `sympy/integrals/meijerint.py` by the internal helper `_meijerint_indefinite_1`. The diagnosis localizes the branch loss around `sympy/integrals/meijerint.py:1736`.

The traced rewrite represents the problem through a Meijer-G expression with argument x^2 and a prefactor $1/x$. In the diagnosed code region, `_meijerint_indefinite_1` extracts powers from the argument and prefactor, substitutes into the Meijer-G antiderivative, applies `hyperexpand`, and then adds a `powdenest(..., polar=True)` simplification:

```

a, b = _get_coeff_exp(g.argument, x)
_, c = _get_coeff_exp(po, x)
...
fac_ = fac * C * x**(1 + c) / b
...
r = hyperexpand(r.subs(t, a*x**b), place=place)
res += powdenest(fac_*r, polar=True)

```

For this integrand, the argument contributes $b = 2$ from x^2 , and the power factor contributes $c = -1$ from $1/x$. The generic substitution–expansion–powerdenesting path returns

$$\frac{x}{\sqrt{1+x^{-2}}} - \operatorname{asinh}\left(\frac{1}{x}\right) + \frac{1}{x\sqrt{1+x^{-2}}}.$$

That expression is branch-correct only where $\sqrt{1+x^{-2}}$ denotes the same branch as $\sqrt{x^2+1}/x$, namely on the positive real branch. On negative real inputs, the missing $\operatorname{sign}(x)$ factor causes the derivative to be $+\sqrt{1+x^{-2}}$ instead of $-\sqrt{1+x^{-2}}$. The diagnosis suggests that this Meijer-G path should avoid applying the rewrite globally when an even power substitution such as x^2 is combined with an odd prefactor such as $1/x$, or should instead return a branch-aware result, preserve polar conditions through simplification, or decline to evaluate without suitable assumptions.

Additional Failing Instances

The independent verification checks the same antiderivative identity at the negative real points $-1, -2, -3, -4, -5, -6$. These are not different formulas; they are pointwise witnesses for the same missing negative-real branch. At every listed point, SymPy’s returned antiderivative differentiates to the positive branch $\sqrt{1+n^2}/n$, while the original integrand at $x = -n$ is the corresponding negative value.

Input / Expression	SymPy behavior	Expected behavior
$x = -1$ in $\sqrt{x^2 + 1}/x$	derivative = $\sqrt{2}$	integrand = $-\sqrt{2}$
$x = -2$ in $\sqrt{x^2 + 1}/x$	derivative = $\sqrt{5}/2$	integrand = $-\sqrt{5}/2$
$x = -3$ in $\sqrt{x^2 + 1}/x$	derivative = $\sqrt{10}/3$	integrand = $-\sqrt{10}/3$
$x = -4$ in $\sqrt{x^2 + 1}/x$	derivative = $\sqrt{17}/4$	integrand = $-\sqrt{17}/4$
$x = -5$ in $\sqrt{x^2 + 1}/x$	derivative = $\sqrt{26}/5$	integrand = $-\sqrt{26}/5$
$x = -6$ in $\sqrt{x^2 + 1}/x$	derivative = $\sqrt{37}/6$	integrand = $-\sqrt{37}/6$

The expected behavior is uniform across this family because, for every positive integer n ,

$$\left. \frac{\sqrt{x^2 + 1}}{x} \right|_{x=-n} = -\frac{\sqrt{n^2 + 1}}{n}.$$

The observed derivative has the opposite sign at each of these negative-real points.

Related Issues Assessment

The bug-finding harness performed a best-effort deduplication check. The artifact reports the verdict as a new instance of a known failure mode: branch sensitivity in indefinite integration with radicals and inverse hyperbolic rewrites is a broadly known kind of problem, especially around transformations that effectively replace $\sqrt{x^2}$ by x rather than $|x|$. However, no public report was found for this exact integrand, returned antiderivative, or pointwise derivative failure at $x = -2$. The available evidence therefore supports treating this as an individually actionable SymPy issue and regression test.

Proposed SymPy Regression Test

```
import pytest
from sympy import diff, integrate, simplify, sqrt, symbols

@pytest.mark.parametrize("n", [1, 2, 3, 4, 5, 6])
def test_integrate_sqrt_x_squared_plus_one_over_x_negative_branch(n):
    x = symbols("x")
    integrand = sqrt(x**2 + 1)/x
    antiderivative = integrate(integrand, x)
```

```
assert simplify((diff(antiderivative, x) - integrand).subs(x, -n)) ==
0
```

A.4.6 Bug 6: Piecewise Left-Hand Limit Returns Right Branch

Minimal Reproducer

```
from sympy import Piecewise, Rational, limit, symbols

x = symbols("x", real=True)
p = Piecewise((0, x < 0), (1, True))

print(limit(p, x, 0, dir="-"))
print(p.subs(x, Rational(-1, 10)))
print(p.subs(x, Rational(1, 10)))
```

Observed Behavior

```
limit(Piecewise((0, x < 0), (1, True)), x, 0, dir='-') = 1
left sample p.subs(x, -1/10) = 0
right sample p.subs(x, 1/10) = 1
```

SymPy 1.14.0 returns 1, the value of the default branch, for the left-hand limit. The same expression evaluates to 0 at a nearby point on the left side, so the returned value is the value from the wrong side of the jump.

Expected Behavior

The left-hand limit should be 0:

$$\lim_{x \rightarrow 0^-} \text{Piecewise}((0, x < 0), (1, \text{True})) = 0.$$

Software-wise, `limit(p, x, 0, dir="-")` should select the branch that is active in a punctured left neighborhood of 0, not the branch that is active at or to the right of the boundary.

Mathematical Explanation

For every real x with $x < 0$, the condition in the first branch is true, so

$$\text{Piecewise}((0, x < 0), (1, \text{True})) = 0.$$

Therefore, for every sufficiently small negative x , the expression is identically 0. The one-sided limit from the left is consequently

$$\lim_{x \rightarrow 0^-} p(x) = 0.$$

The returned value 1 is not an equivalent representation or a harmless branch choice. It is the right/default branch value: for instance

$$p(-1/10) = 0, \quad p(1/10) = 1.$$

A left-hand limit at this jump must be determined by the left-neighborhood values, not by substituting the boundary point into the branch condition.

Source-Level Diagnosis

The diagnosis identifies the source file `sympy/functions/elementary/piecewise.py`. The relevant method and traced call path are:

```
limit(p, x, 0, dir="-")
Limit(...).doit(deep=False)
Piecewise._eval_as_leading_term(x, logx, cdir)
```

The surrounding limit code in `sympy/series/limits.py` constructs a `Limit` object and evaluates it via `doit`. For a finite directed limit, the limit engine shifts the point of approach to 0 and records the left-hand direction as `cdir = -1`. It then asks the `Piecewise` expression for its leading term.

The relevant method receives the direction parameter but ignores it:

```
def _eval_as_leading_term(self, x, logx, cdir):
    for e, c in self.args:
        if c == True or c.subs(x, 0) == True:
            return e.as_leading_term(x)
```

For the condition $x < 0$, direct substitution at the boundary gives $(x < 0)|_{x=0}$, which is false. The method therefore skips the left branch even though `cdir = -1` means the approach is through negative values of the shifted variable. It then reaches the default `True` branch and returns the leading term of the expression 1, which the limit engine reports as the left-hand limit.

A plausible fix direction supported by the diagnosis is to make this `Piecewise` leading-term method direction-aware. For relational conditions involving the limit variable, it should test which branch holds in a punctured neighborhood determined by `cdir`, for example through a signed dummy substitution or an interval/set containment check, rather than by substituting the exact boundary point.

Additional Failing Instances

The independent verification checks the representative case together with five more constant jumps of the same form:

$$p_{a,b}(x) = \text{Piecewise}((a, x < 0), (b, \text{True})).$$

For each pair (a, b) , every sufficiently small negative x satisfies $x < 0$, so the expected left-hand limit is a . SymPy instead returns b , the default branch value.

Input / Expression	SymPy behavior	Expected behavior
$p_{0,1}$	returns 1, the default branch	returns 0, the left branch
$p_{2,5}$	returns 5, the default branch	returns 2, the left branch
$p_{-3,7}$	returns 7, the default branch	returns -3, the left branch
$p_{4,-2}$	returns -2, the default branch	returns 4, the left branch
$p_{11,13}$	returns 13, the default branch	returns 11, the left branch
$p_{-8,-1}$	returns -1, the default branch	returns -8, the left branch

The expected behavior is the same across the family because the condition $x < 0$ is true throughout a punctured left neighborhood of the limit point. The right/default branch may determine values at $x = 0$ or for $x > 0$, but it does not determine the left-hand limit.

Related Issues Assessment

The bug-finding harness initially classified this as a new failure mode with no public precedent, but that classification was incorrect. This is a known, long-standing problem in SymPy. It is the same root cause described in the open issue [#23836](#) (“Incorrect results for limits of Piecewise at discontinuity”), which states that the value of the `Piecewise` at the boundary should not be considered by `limit`, and it is a sibling of the open issue [#27236](#) (“bug with `limit()` function”), where one-sided limits of a discontinuous `Piecewise` return the wrong branch. There are also open pull requests targeting the precise function diagnosed above (`Piecewise._eval_as_leading_term` ignoring the direction `cdir`): [#29589](#), [#29538](#), and [#29920](#). The closest related public material is therefore not merely subsystem-level: these issues and PRs describe this exact failure. The bug remains a clear, reproducible instance of this known family, and the focused regression test below guards branch selection in directed limits of `Piecewise` expressions.

Proposed SymPy Regression Test

```
import pytest

from sympy import Piecewise, limit, symbols

@pytest.mark.parametrize(
    "left_value, default_value",
    [(0, 1), (2, 5), (-3, 7), (4, -2), (11, 13), (-8, -1)],
)
```



```
def test_pieewise_left_hand_limit_uses_left_branch(left_value,
    default_value):
    x = symbols("x", real=True)
    p = Piecewise((left_value, x < 0), (default_value, True))
    assert limit(p, x, 0, dir="-") == left_value
```

A.4.7 Bug 7: dsolve returns an invalid branch for f prime equals sqrt f squared

Minimal Reproducer

```
from sympy import Eq, Function, N, diff, dsolve, simplify, sqrt, symbols

x = symbols("x")
f = Function("f")
C1 = symbols("C1")

ode = Eq(diff(f(x), x), sqrt(f(x)**2))
sol = dsolve(ode)

rhs = sol[0].rhs.subs(C1, 1)
residual = diff(rhs, x) - sqrt(rhs**2)

print("dsolve result:", sol)
print("chosen returned branch with C1=1:", rhs)
print("residual:", simplify(residual))
print("residual at x=0:", N(residual.subs(x, 0), 50))
```

Observed Behavior

```
dsolve result: [Eq(f(x), C1*exp(-x)), Eq(f(x), C1*exp(x))]  
chosen returned branch with C1=1: exp(-x)  
residual: -sqrt(exp(-2*x)) - exp(-x)  
residual at x=0: -2.0000000000000000000000000000000000000000000000000000000
```

SymPy returns an unrestricted decreasing exponential branch. Setting the arbitrary constant in that branch to 1 gives $f(x) = e^{-x}$, but the residual of the original differential equation is nonzero. At $x = 0$, the left-hand side is -1 while the right-hand side is 1 .

Expected Behavior

The returned solution set should not include an unrestricted branch $f(x) = C_1 e^{-x}$. For real-valued solutions, any decreasing exponential branch must carry

the sign condition $C_1 < 0$; equivalently, the solver should leave the branch-sensitive solution implicit or return a branch-conditioned answer. Positive-constant decreasing exponentials such as e^{-x} are not solutions of $f' = \sqrt{f^2}$.

Mathematical Explanation

For real-valued f ,

$$\sqrt{f(x)^2} = |f(x)|.$$

Therefore the differential equation is branch-sensitive:

$$f'(x) = |f(x)|.$$

On a positive branch, this reduces to $f' = f$, so the nonzero solutions are $f(x) = Ce^x$ with $C > 0$. On a negative branch, this reduces to $f' = -f$, so the nonzero solutions are $f(x) = Ce^{-x}$ with $C < 0$. The zero solution is also valid.

The returned expression $C_1 e^{-x}$ is not wrong merely because of a different representation or a missing integration constant; it is wrong because SymPy returns it with C_1 unrestricted. Substituting $C_1 = 1$ gives

$$f(x) = e^{-x}, \quad f'(x) = -e^{-x}, \quad \sqrt{f(x)^2} = \sqrt{e^{-2x}} = e^{-x}$$

on the real line, and hence

$$f'(x) - \sqrt{f(x)^2} = -2e^{-x},$$

which is not identically zero.

Source-Level Diagnosis

The diagnosis identifies the relevant source-level path in the ODE solver. In `sympy/solvers/ode/single.py`, the `Separable` hint builds an implicit separable solution equivalent to

$$\int^{f(x)} \frac{dy}{\sqrt{y^2}} = C_1 + \int^x 1 \, dx.$$

During simplification, `odesimp` in `sympy/solvers/ode/ode.py` calls `_handle_Integral`, which evaluates the left integral to a branch-sensitive expression involving $f(x) \log(f(x)) / \sqrt{f(x)^2}$. The traced call path then reaches the explicit solving step in `odesimp`, reported around `sympy/solvers/ode/ode.py:1666`.

The key problematic rule is that `odesimp` explicitly solves the branch-sensitive implicit equation without retaining the sign condition that distinguishes the positive and negative real branches:

```
eq = _handle_Integral(eq, func, hint)
...
eqsol = solve(eq, func, force=True, rational=False if floats else None)
...
eq = [Eq(f(x), _expand(t)) for t in eqsol]
```

This produces both C_1e^{-x} and C_1e^x as explicit branches with an unrestricted integration constant. The decreasing exponential is valid only on a negative branch of f , so the unrestricted return value falsely permits positive constants. A plausible fix direction, supported by the diagnosis, is to avoid forcing branch-sensitive implicit ODE solutions into unconstrained explicit branches unless the necessary sign conditions can be attached. A post-solve residual validation step, similar in spirit to checking returned ODE solutions, would also reject the positive-constant decreasing branch.

Additional Failing Instances

The independent verification exercises the same returned branch with positive constants 1 through 6. In each case SymPy's unrestricted C_1e^{-x} branch admits $C_1 = c > 0$, producing $f(x) = ce^{-x}$. The residual at $x = 0$ is

$$f'(0) - \sqrt{f(0)^2} = -c - \sqrt{c^2} = -2c,$$

so every tested positive constant gives a concrete false solution. The expected behavior is the same across the family: the decreasing branch should either be absent for positive constants or be returned with a condition such as $C_1 < 0$.

Input / Expression	SymPy behavior	Expected behavior
$C_1 = 1$ in C_1e^{-x}	admits e^{-x} , residual at 0 is -2	reject this branch or require $C_1 < 0$
$C_1 = 2$ in C_1e^{-x}	admits $2e^{-x}$, residual at 0 is -4	reject this branch or require $C_1 < 0$
$C_1 = 3$ in C_1e^{-x}	admits $3e^{-x}$, residual at 0 is -6	reject this branch or require $C_1 < 0$
$C_1 = 4$ in C_1e^{-x}	admits $4e^{-x}$, residual at 0 is -8	reject this branch or require $C_1 < 0$
$C_1 = 5$ in C_1e^{-x}	admits $5e^{-x}$, residual at 0 is -10	reject this branch or require $C_1 < 0$
$C_1 = 6$ in C_1e^{-x}	admits $6e^{-x}$, residual at 0 is -12	reject this branch or require $C_1 < 0$

Related Issues Assessment

The bug-finding harness performed a best-effort deduplication check. The closest related public material concerns the general branch/sign issue for $\sqrt{x^2}$, including SymPy documentation that $\sqrt{x^2}$ is not generally equal to x and general references noting that $\sqrt{x^2} = |x|$ over the reals. Focused searches did not identify a public report for this specific `dsolve` input, the returned invalid C_1e^{-x} branch, or the residual failure above. The deduplication verdict is therefore a new instance of a known failure mode: the broad square-root branch issue

is known, but this exact ODE solver counterexample is previously unreported in the available evidence and remains individually actionable as an issue and regression test.

Proposed SymPy Regression Test

```
import pytest

from sympy import Eq, Function, N, diff, dsolve, sqrt, symbols

@pytest.mark.parametrize("c", [1, 2, 3, 4, 5, 6])
def
    test_dsolve_sqrt_f_squared_branches_satisfy_ode_for_positive_constants
    (c):
    x = symbols("x")
    f = Function("f")
    ode = Eq(diff(f(x), x), sqrt(f(x)**2))
    sol = dsolve(ode)

    for equation in sol:
        rhs = equation.rhs
        constants = [s for s in rhs.free_symbols if s != x]
        concrete_rhs = rhs.subs({constant: c for constant in constants})
        residual_at_zero = N((diff(concrete_rhs, x) - sqrt(concrete_rhs
**2)).subs(x, 0), 50)
        assert abs(complex(residual_at_zero)) < 1e-40
```

A.4.8 Bug 8: K-Weighted Alternating Binomial Sum Misses the $n = 1$ Case

Minimal Reproducer

```
from sympy import binomial, summation, symbols

n = symbols("n", integer=True, nonnegative=True)
k = symbols("k", integer=True)

closed = summation((-1)**k*k*binomial(n, k), (k, 0, n))
direct_n1 = sum((-1)**j*j*binomial(1, j) for j in range(0, 2))

print("closed form =", closed)
print("closed at n=1 =", closed.subs(n, 1))
print("direct finite sum at n=1 =", direct_n1)
print("difference =", closed.subs(n, 1) - direct_n1)
```

Observed Behavior

```
closed form = 0
closed at n=1 = 0
direct finite sum at n=1 = -1
difference = 1
```

SymPy returns the unconditional closed form 0. Substituting $n = 1$ into that result gives 0, while direct evaluation of the finite sum at $n = 1$ gives -1 . The returned scalar therefore loses a parameter value allowed by the declared assumption $n \in \mathbb{Z}_{\geq 0}$.

Expected Behavior

For a nonnegative integer symbol n , the result should preserve the exceptional $n = 1$ value. A correct result for the representative sum is

$$\text{Piecewise}((-1, \text{Eq}(n, 1)), (0, \text{Ne}(n, 1))), \quad n \in \mathbb{Z}_{\geq 0},$$

or any equivalent expression that evaluates to -1 at $n = 1$ and 0 at all other nonnegative integer values of n .

Mathematical Explanation

Differentiate the binomial identity

$$(1 + t)^n = \sum_{k=0}^n \binom{n}{k} t^k$$

with respect to t . This gives

$$n(1 + t)^{n-1} = \sum_{k=0}^n k \binom{n}{k} t^{k-1}.$$

Multiplying by t and setting $t = -1$ yields

$$\sum_{k=0}^n (-1)^k k \binom{n}{k} = -n(1 - 1)^{n-1}.$$

For $n > 1$, this is 0. For $n = 1$, however, the differentiated identity must be evaluated as the original finite sum:

$$\sum_{k=0}^1 (-1)^k k \binom{1}{k} = 0 \binom{1}{0} - 1 \binom{1}{1} = -1.$$

The case $n = 0$ is also 0, since the only summand contains the factor $k = 0$. Thus the scalar 0 is not an equivalent simplification over the declared nonnegative-integer domain; it has the wrong value at $n = 1$.

Source-Level Diagnosis

The diagnosis locates the failure in SymPy's concrete summation path. The public call constructs a `Sum` and calls `doit`. The relevant source files are:

```
sympy/concrete/summations.py  
sympy/concrete/gosper.py
```

Since the range length is symbolic, evaluation reaches `eval_sum` and then `eval_sum_symbolic`. The traced call path reaches the Gosper branch around `sympy/concrete/summations.py` lines 1236–1256, which calls `gosper_sum`.

For the summand $(-1)^k k \binom{n}{k}$, the traced Gosper certificate is

$$g = -\frac{k-1}{n-1}.$$

This certificate has a pole at the parameter value $n = 1$. The diagnosis identifies the immediate faulty endpoint calculation in `sympy/concrete/gosper.py`:

```
result = (f*(g + 1)).subs(k, b) - (f*g).subs(k, a)
```

`gosper_sum` substitutes the symbolic endpoints and simplifies the generic telescoping result to the scalar 0, without recording that the certificate is invalid at $n = 1$. The caller has logic to wrap some results in `Piecewise` when visible denominators remain in `Mul` or `Add` results, but here the endpoint expression has already simplified to 0. As a result, the excluded parameter value is no longer visible to `eval_sum_symbolic`, and the generic value becomes the final answer.

The suggested fix direction is to track parameter denominators introduced by `gosper_term` or `gosper_sum`, and either emit a conditional result or fall back to direct evaluation at excluded parameter values. For this example, the final result needs an $n = 1$ branch.

Additional Failing Instances

The independent verification checks the same endpoint failure for

$$\sum_{k=0}^n (-1)^k k^p \binom{n}{k}, \quad p = 1, 2, 3, 4, 5, 6.$$

For each listed power, SymPy returns a closed form whose value at $n = 1$ is 0. The direct finite sum at $n = 1$ is the same in all cases with $p \geq 1$, because the $k = 0$ term is $0^p \binom{1}{0} = 0$ and the $k = 1$ term is $(-1)1^p \binom{1}{1} = -1$.

Input / Expression	SymPy behavior	Expected behavior
$\sum_{k=0}^n (-1)^k k \binom{n}{k}$, evaluated at $n = 1$	Closed form gives 0	Direct sum gives -1
$\sum_{k=0}^n (-1)^k k^2 \binom{n}{k}$, evaluated at $n = 1$	Closed form gives 0	Direct sum gives -1
$\sum_{k=0}^n (-1)^k k^3 \binom{n}{k}$, evaluated at $n = 1$	Closed form gives 0	Direct sum gives -1
$\sum_{k=0}^n (-1)^k k^4 \binom{n}{k}$, evaluated at $n = 1$	Closed form gives 0	Direct sum gives -1
$\sum_{k=0}^n (-1)^k k^5 \binom{n}{k}$, evaluated at $n = 1$	Closed form gives 0	Direct sum gives -1
$\sum_{k=0}^n (-1)^k k^6 \binom{n}{k}$, evaluated at $n = 1$	Closed form gives 0	Direct sum gives -1

Related Issues Assessment

The bug-finding harness performed a best-effort deduplication check. Besides general documentation and background for Gosper summation, the open issue [sympy/sympy#29731](#) (“A summation that evaluates to 0 even though it can be nonzero depending on the variable”, created 2026-05-02) is a near-exact public report of this same behavior: its reproducer returns the scalar 0 for an (-1) -weighted i^k binomial sum that is nonzero at the singular parameter value, with the same `gosper_sum` lost-endpoint-exception root cause. That issue predates this bundle, so the deduplication verdict is a new instance of an already-reported failure family: symbolic Gosper summation can require parameter conditions, and this candidate contributes the explicit $(-1)^k k \binom{n}{k}$ instantiation, the missing $n = 1$ branch, and the k^p generalization, all of which remain individually actionable as a cross-link to [#29731](#) and as a regression test.

Proposed SymPy Regression Test

```
import pytest
from sympy import binomial, summation, symbols

n = symbols("n", integer=True, nonnegative=True)
k = symbols("k", integer=True)
CASES = [1, 2, 3, 4, 5, 6]

@pytest.mark.parametrize("power", CASES)
def test_weighted_alternating_binomial_sum_preserves_n1_exception(power):
    closed = summation((-1)**k*k**power*binomial(n, k), (k, 0, n))
    direct = sum((((1)**j)*(j**power)*binomial(1, j) for j in range(2))
    assert closed.subs(n, 1) == direct
```

A.4.9 Bug 9: Finite Product Masks an Undefined Factor as Zero

Minimal Reproducer

```
from sympy import pi, product, sin, symbols

n = symbols("n")
term = sin(pi*n)/(n - 1)

print("term =", term)
print("product(term, (n, 0, 2)) =", product(term, (n, 0, 2)))
print("term values =", [term.subs(n, i) for i in [0, 1, 2]])
```

Observed Behavior

```
term = sin(pi*n)/(n - 1)
product(term, (n, 0, 2)) = 0
term values = [0, nan, 0]
```

On SymPy 1.14.0, the finite product is evaluated as 0, even though direct substitution of the individual factors shows that the middle factor is nan. The zero endpoint factors do not make a product containing an undefined factor well-defined.

Expected Behavior

The product should evaluate to nan, or otherwise remain unevaluated with the singular factor exposed. It should not collapse to 0.

Mathematical Explanation

The product requested from SymPy is

$$\prod_{n=0}^2 \frac{\sin(\pi n)}{n-1}.$$

The three raw factors are

$$\frac{\sin(0)}{-1} = 0, \quad \frac{\sin(\pi)}{0} = \frac{0}{0}, \quad \frac{\sin(2\pi)}{1} = 0.$$

The middle factor is undefined. In SymPy's own arithmetic, direct multiplication of the substituted factors gives nan, not 0. Therefore the returned value is not an equivalent simplification, a harmless choice of representation, or a convention for an empty product; it loses the domain failure at $n = 1$.

Source-Level Diagnosis

The diagnosis locates the failure in `sympy/concrete/products.py`, inside `Product.doit`, with the relevant code around line 256. The public call `product` constructs a `Product` and calls `Product.doit`. At the start of `doit`, definite product variables are replaced by dummies with assumptions inherited from the concrete limits before the product evaluator runs. For the limits 0 and 2, the helper imported from `sympy/concrete/summations.py` creates an integer, nonnegative, real dummy.

```
for xab in self.limits:
    d = _dummy_with_inherited_properties_concrete(xab)
    if d:
        reps[xab[0]] = d
...
did = self.xreplace(reps).doit(**hints)
```

After replacing n by this assumption-enriched dummy, $\sin(\pi n)$ simplifies as zero for an integer index. Consequently, the whole product function becomes 0 before `_eval_product` sees the original denominator $n - 1$ and the singular point $n = 1$. The diagnosis notes that the downstream direct product evaluator in `sympy/concrete/products.py` would produce nan for the original raw factors, but it is bypassed because the pre-evaluated term is already zero. A plausible fix direction identified by the diagnosis is to check original finite-range factors for undefined values before accepting an identically zero product function, and more generally to avoid allowing strengthened index assumptions to mask denominator singularities in concrete products.

Additional Failing Instances

The independent verification covers a shifted family

$$\prod_{n=a-1}^{a+1} \frac{\sin(\pi n)}{n-a}, \quad a \in \{1, 2, 3, 4, 5, 6\}.$$

For every listed a , the factor at $n = a$ is $0/0$, while the two adjacent integer factors have zero numerator. The expected result is therefore nan across the family, because every product contains an undefined factor.

Input / Expression	SymPy behavior	Expected behavior
$\prod_{n=0}^2 \sin(\pi n)/(n-1)$	returns 0	nan
$\prod_{n=1}^3 \sin(\pi n)/(n-2)$	returns 0	nan
$\prod_{n=2}^4 \sin(\pi n)/(n-3)$	returns 0	nan
$\prod_{n=3}^5 \sin(\pi n)/(n-4)$	returns 0	nan
$\prod_{n=4}^6 \sin(\pi n)/(n-5)$	returns 0	nan
$\prod_{n=5}^7 \sin(\pi n)/(n-6)$	returns 0	nan

Related Issues Assessment

The deduplication check identified only general concrete-product documentation as related background and did not find a public issue, pull request, Stack Overflow question, mailing-list thread, or release-note entry for this exact finite product or for the specific failure where integer-index assumptions simplify $\sin(\pi n)$ to zero before the denominator singularity is checked. The verdict is a new failure mode with no public precedent. The bug remains individually actionable because the reproducer is minimal, the affected call is in SymPy's public concrete-product API, and the proposed regression test pins the exact finite-range behavior that should be preserved.

Proposed SymPy Regression Test

```
import pytest
from sympy import Mul, nan, pi, product, sin, symbols

@pytest.mark.parametrize("a", [1, 2, 3, 4, 5, 6])
def test_finite_product_preserves_undefined_factor_before_zero_collapse(a):
    n = symbols("n")
    term = sin(pi*n)/(n - a)
    values = [term.subs(n, i) for i in range(a - 1, a + 2)]

    assert nan in values
    assert Mul(*values) is nan
    assert product(term, (n, a - 1, a + 1)) is nan
```

A.4.10 Bug 10: Principal Cube-Root Inequality Includes Negative Reals

Minimal Reproducer

```

from sympy import S, N, symbols
from sympy.solvers.inequalities import solve_univariate_inequality

x = symbols("x")
sol = solve_univariate_inequality(x**(S(1)/3) <= 1, x, relational=False)

print("solution:", sol)
print("contains_minus_one:", sol.contains(-1))
print("lhs_at_minus_one:", N((-1)**(S(1)/3), 30))

```

Observed Behavior

```

solution: Interval(-oo, 1)
contains_minus_one: True
lhs_at_minus_one: 0.5 + 0.866025403784438646763723170753*I

```

SymPy reports that the solution set is $(-\infty, 1]$. In particular, the reported set contains $x = -1$, even though SymPy's principal value of $(-1)^{1/3}$ is non-real. Such a point cannot satisfy a real-valued inequality of the form $x^{1/3} \leq 1$.

Expected Behavior

The mathematically correct solution over the real line is

$$[0, 1].$$

More generally, for the parametrized family checked in the artifact,

$$\text{solve_univariate_inequality}\left(x^{1/3} \leq m, x, \text{relational=False}\right) = [0, m^3]$$

for the positive integer values $m = 1, \dots, 6$.

Mathematical Explanation

SymPy's expression $x^{1/3}$, represented as $x**(S(1)/3)$, uses the principal branch of the power function. For a negative real input $x = -a$ with $a > 0$,

$$(-a)^{1/3} = \exp\left(\frac{\log(a) + i\pi}{3}\right) = a^{1/3} \left(\frac{1}{2} + \frac{\sqrt{3}}{2}i\right),$$

which is not real. Therefore the real-valued domain of the left-hand side is not all of $\mathbb{R}$; for this inequality it is $x \geq 0$.

On that domain, the principal cube root agrees with the ordinary nonnegative real cube root and is monotone increasing. Thus

$$x^{1/3} \leq 1 \iff 0 \leq x \leq 1.$$

The returned interval $(-\infty, 1]$ is not an equivalent representation or a harmless branch convention: it contains points at which the left-hand side is complex, so the ordered real comparison is not satisfied.

Source-Level Diagnosis

The diagnosis identifies the primary source-level issue in `sympy/calculus/util.py`, in the function `continuous_domain`. The public call enters `solve_univariate_inequality` in `sympy/solvers/inequalities.py`. For the representative input, the solver forms $e = x^{1/3} - 1$, obtains the equality root $x = 1$, and asks `continuous_domain` for the real continuity domain of that expression. The interval-building code in `solve_univariate_inequality` then tests sample points between critical points and accepts whole intervals whose samples satisfy the inequality.

The problematic rule reported in `continuous_domain` treats rational powers with odd denominator as real-valued over all real bases:

```
if atom.exp.is_rational and den.is_odd:
    pass
```

That rule matches `real_root`-style semantics, but it is too broad for principal `Pow` semantics. For $x^{1/3} - 1$, it causes `continuous_domain` to return all real numbers rather than restricting the expression to $x \geq 0$. With the domain incorrectly set to $\mathbb{R}$, the inequality solver samples the interval $(-\infty, 1]$; the selected sample point satisfies the inequality, so the solver appends the entire interval $(-\infty, 1]$. No later filtering removes the negative inputs where the principal fractional power is complex.

The suggested fix direction in the diagnosis is to make `continuous_domain` distinguish principal `Pow` from real-root semantics. For rational non-integer powers of a real expression, a plausible fix is to require the base to be nonnegative unless SymPy can prove the principal value remains real on a larger set. The inequality solver should also intersect interval results with the real-valued domain of the relational expression.

Additional Failing Instances

The independent verification checks the representative case together with the positive integer parameters $m = 1, \dots, 6$. In each case, SymPy returns an interval of the form $(-\infty, m^3]$, which includes negative real inputs such as -1 . The expected behavior is uniformly $[0, m^3]$, because the principal power $x^{1/3}$ is real-valued only for $x \geq 0$, and on that domain $x^{1/3} \leq m$ is equivalent to $x \leq m^3$ for these positive m .

Input / Expression	SymPy behavior	Expected behavior
<code>solve_univariate_inequality($x^{1/3} \leq 1$)</code> 1)	<code>Interval(-oo, 1)</code>	<code>Interval(0, 1)</code>
<code>solve_univariate_inequality($x^{1/3} \leq 8$)</code> 2)	<code>Interval(-oo, 8)</code>	<code>Interval(0, 8)</code>
<code>solve_univariate_inequality($x^{1/3} \leq 27$)</code> 3)	<code>Interval(-oo, 27)</code>	<code>Interval(0, 27)</code>
<code>solve_univariate_inequality($x^{1/3} \leq 64$)</code> 4)	<code>Interval(-oo, 64)</code>	<code>Interval(0, 64)</code>
<code>solve_univariate_inequality($x^{1/3} \leq 125$)</code> 5)	<code>Interval(-oo, 125)</code>	<code>Interval(0, 125)</code>
<code>solve_univariate_inequality($x^{1/3} \leq 216$)</code> 6)	<code>Interval(-oo, 216)</code>	<code>Interval(0, 216)</code>

Related Issues Assessment

The bug-finding harness performed a best-effort deduplication check and classified this as a new instance of a known failure mode. The closest public material identified was SymPy documentation distinguishing principal roots from `real_root`, together with broader public awareness that principal-root domain handling can be subtle. The available evidence did not identify a clear public duplicate for this exact `solve_univariate_inequality` input or for the exact wrong interval `Interval(-oo, 1)`. This case is therefore individually actionable as a concrete regression test and issue, even though the general semantic pitfall is known.

Proposed SymPy Regression Test

```
import pytest
from sympy import Interval, S, symbols
from sympy.solvers.inequalities import solve_univariate_inequality

@pytest.mark.parametrize("m", [1, 2, 3, 4, 5, 6])
def test_fractional_power_inequality_real_domain(m):
    x = symbols("x")
    assert solve_univariate_inequality(x**(S(1)/3) <= m, x, relational=False) == Interval(0, m**3)
```

A.4.11 Bug 11: `function_range` returns `EmptySet` for `sqrt(x**2)` over the reals

Minimal Reproducer

```

from sympy import S, sqrt, symbols
from sympy.calculus.util import function_range

x = symbols("x")
r = function_range(sqrt(x**2), x, S.Reals)

print("range:", r)
print("values:", sqrt((-2)**2), sqrt(0**2), sqrt(3**2))
print("contains 0:", r.contains(0))
print("contains 2:", r.contains(2))

```

Observed Behavior

```

range: EmptySet
values: 2 0 3
contains 0: False
contains 2: False

```

SymPy reports that the range is `EmptySet`. This is not merely an alternative representation of the same set: the output excludes values such as 0 and 2, even though the expression directly attains those values at $x = 0$ and $x = -2$, respectively.

Expected Behavior

The correct result is $\text{Interval}(0, \infty)$, represented in SymPy as `Interval(0, oo)`. The call

$$\text{function_range}\left(\sqrt{x^2}, x, \mathbb{R}\right)$$

should return the set of all nonnegative real numbers.

Mathematical Explanation

For real x ,

$$\sqrt{x^2} = |x|.$$

The absolute-value function is continuous on $\mathbb{R}$, has a global minimum 0 at $x = 0$, and is unbounded above. Hence

$$\{\sqrt{x^2} : x \in \mathbb{R}\} = \{|x| : x \in \mathbb{R}\} = [0, \infty).$$

Every nonnegative value y is attained, for example by taking $x = y$. The reported value $\emptyset$ is therefore a genuine loss of the whole range, not a harmless branch convention, simplification choice, or equivalent set representation.

Source-Level Diagnosis

The diagnosis locates the failure in `sympy/calculus/util.py`, inside `function_range`, especially the logic around lines 253–281 and the critical-point search at line 260. The traced call path enters `function_range`, computes

`continuous_domain(sqrt(x**2), x, S.Reals)`, and obtains the full real interval. The algorithm then samples the interval endpoint limits and solves $f'(x) = 0$ with `solveset(f.diff(x), x, interval)`.

For $f(x) = \sqrt{x^2}$, both endpoint limits are ∞ . The derivative expression is `sqrt(x**2)/x`, effectively $|x|/x$ on the reals, which is undefined at the cusp $x = 0$. Consequently, the global minimum occurs at a point where the derivative is undefined rather than zero. Since `function_range` samples derivative-zero points but does not also sample interior derivative singularities where the original function remains continuous, $x = 0$ is not added as a candidate value.

With only the infinite endpoint values available, the final interval construction uses $\inf = \infty$ and $\sup = \infty$, which collapses to `EmptySet`. The source-level behavior therefore misses the finite minimum that determines the lower endpoint of the true range. A plausible fix direction identified by the diagnosis is to include derivative singularities lying inside the original function's continuous domain as candidate points, while intersecting such points with the original continuous domain to avoid spurious candidates.

Additional Failing Instances

The same failure appears for shifted cusp expressions of the form $\sqrt{(x+a)^2}$. Over the reals,

$$\sqrt{(x+a)^2} = |x+a|,$$

so each case has the same range $[0, \infty)$, with the minimum attained at $x = -a$. The independent verification checks the representative case and five shifted variants, comparing SymPy's range against `Interval(0, oo)` and direct sample values computed from the real square-root formula.

Input / Expression	SymPy behavior	Expected behavior
<code>function_range(sqrt(x**2), x, R)</code>	<code>EmptySet</code>	<code>Interval(0, oo)</code>
<code>function_range(sqrt((x+1)**2), x, R)</code>	<code>EmptySet</code>	<code>Interval(0, oo)</code>
<code>function_range(sqrt((x-1)**2), x, R)</code>	<code>EmptySet</code>	<code>Interval(0, oo)</code>
<code>function_range(sqrt((x+2)**2), x, R)</code>	<code>EmptySet</code>	<code>Interval(0, oo)</code>
<code>function_range(sqrt((x-2)**2), x, R)</code>	<code>EmptySet</code>	<code>Interval(0, oo)</code>
<code>function_range(sqrt((x+3)**2), x, R)</code>	<code>EmptySet</code>	<code>Interval(0, oo)</code>

Related Issues Assessment

The bug-finding harness performed a best-effort deduplication check. The closest public material found was documentation for `function_range`, the real-domain refinement $\sqrt{x^2} \mapsto |x|$, and the elementary-function documentation for `sqrt` and `Abs`. No public duplicate was identified for the exact

`function_range(sqrt(x**2), x, S.Reals)` returning `EmptySet` symptom. The underlying failure family is, however, publicly known: the root cause is `function_range`'s critical-point search missing the non-differentiable cusp of $\sqrt{x^2} = |x|$ at $x = 0$, the same `Abs`-cusp mechanism documented in [sympy/sympy#16469](#) (`function_range(abs(x), x, S.Reals)` failing; correct only with `Symbol("x", real=True)`), [#21956](#) ("Unable to find critical points for `Abs(f(x))`"), and [#20101](#) (maximum of an `Abs` expression failing). The deduplication verdict is therefore a specific new instance of this known family rather than a wholly new failure mode with no public precedent. The case remains individually actionable because it is a small reproducible failure in `calculus.util.function_range` with a concise regression test.

Proposed SymPy Regression Test

```
import pytest

from sympy import Interval, S, oo, sqrt, symbols
from sympy.calculus.util import function_range

@pytest.mark.parametrize("a", [0, 1, -1, 2, -2, 3])
def test_function_range_sqrt_shifted_square_over_reals(a):
    x = symbols("x")

    assert function_range(sqrt((x + a)**2), x, S.Reals) == Interval(0, oo
    )
```

A.4.12 Bug 12: Inverse Fourier transform drops `Abs` in the Lorentzian pair

Minimal Reproducer

```
from sympy import exp, inverse_fourier_transform, pi, simplify, symbols

t, w = symbols("t w", real=True)
F = 2/(1 + 4*pi**2*w**2)
inv = inverse_fourier_transform(F, w, t)

print("inverse =", inv)
print("at t=-1:", inv.subs(t, -1))
print("expected at t=-1:", exp(-1))
print("difference:", simplify(inv.subs(t, -1) - exp(-1)))
```

Observed Behavior


```

SymPy version: 1.14.0
inverse = exp(-t)
at t=-1: E
expected at t=-1: exp(-1)
difference: 2*sinh(1)

```

SymPy returns e^{-t} as an unconditional inverse Fourier transform. This expression agrees with the correct inverse only on the nonnegative real half line. At $t = -1$, the returned expression evaluates to e , while the expected value is e^{-1} .

Expected Behavior

The inverse Fourier transform should be

$$\text{inverse_fourier_transform}\left(\frac{2}{1 + 4\pi^2 w^2}, w, t\right) = e^{-|t|}.$$

Equivalently, for real t , the result should be the even function $\exp(-\text{Abs}(t))$, not the one-sided branch $\exp(-t)$.

Mathematical Explanation

With SymPy's Fourier convention,

$$\mathcal{F}\{f\}(w) = \int_{-\infty}^{\infty} f(t) e^{-2\pi i w t} dt, \quad \mathcal{F}^{-1}\{F\}(t) = \int_{-\infty}^{\infty} F(w) e^{2\pi i w t} dw.$$

The standard Lorentzian transform pair is

$$\int_{-\infty}^{\infty} e^{-a|t|} e^{-2\pi i w t} dt = \frac{2a}{a^2 + 4\pi^2 w^2} \quad (a > 0).$$

For $a = 1$, the inverse transform of $2/(1 + 4\pi^2 w^2)$ is therefore $e^{-|t|}$. The expression e^{-t} is not an equivalent representation of $e^{-|t|}$: for $t < 0$,

$$e^{-t} = e^{|t|} \neq e^{-|t|}.$$

The discrepancy is not a missing multiplicative constant or a harmless branch choice. It changes the decay on the negative half line into exponential growth.

Source-Level Diagnosis

The diagnosis identifies the relevant implementation as `sympy/integrals/transforms.py`, specifically `_fourier_transform` around lines 945–961. The public call `inverse_fourier_transform(F, w, t)` constructs an inverse transform object. Its compute method follows the shared Fourier-transform path and then calls `_fourier_transform` with the inverse-Fourier parameters. That helper evaluates the real-axis kernel integral

$$\int_{-\infty}^{\infty} F(w) e^{2\pi i w t} dw.$$

For the reproducer, the raw integral result is a `Piecewise` value whose first branch is a closed form valid only under the condition $t > 0$, with an unevaluated integral as the fallback branch. The diagnosed problematic rule is that `_fourier_transform` accepts a `Piecewise` integral result by taking only the first branch and returning that branch expression together with its condition:

```
if F.is_Piecewise:
    F, cond = F.args[0]
    return _simplify(F), cond
```

In this case, the first branch simplifies to $\exp(-t)$ and its condition is $t > 0$. The public Fourier transform wrapper uses `noconds=True` by default, so the condition is discarded before the result is returned to the caller. The source-level failure is therefore a conditional branch being promoted to an unconditional transform result. A plausible fix direction, supported by the diagnosis, is to avoid collapsing a conditional `Piecewise` transform integral to its first branch unless that condition is globally valid; otherwise SymPy should preserve the conditional result, compute the missing branch, or leave the transform unevaluated when `noconds=True` would hide a necessary condition.

Additional Failing Instances

The same error appears for the parametrized Lorentzian family

$$F_a(w) = \frac{2a}{a^2 + 4\pi^2 w^2}, \quad a > 0.$$

Mathematically, this is the Fourier transform of $e^{-a|t|}$, so every inverse transform in the family should be even in t . The verification file checks $a = 1, 2, 3, 4, 5, 6$ at $t = -1$; in each case the returned positive- t branch gives e^a rather than e^{-a} .

Input / Expression	SymPy behavior	Expected behavior
$2/(1 + 4\pi^2 w^2)$	returns e^{-t} ; at $t = -1$, e	$e^{- t }$; at $t = -1$, e^{-1}
$4/(4 + 4\pi^2 w^2)$	returns e^{-2t} ; at $t = -1$, e^2	$e^{-2 t }$; at $t = -1$, e^{-2}
$6/(9 + 4\pi^2 w^2)$	returns e^{-3t} ; at $t = -1$, e^3	$e^{-3 t }$; at $t = -1$, e^{-3}
$8/(16 + 4\pi^2 w^2)$	returns e^{-4t} ; at $t = -1$, e^4	$e^{-4 t }$; at $t = -1$, e^{-4}
$10/(25 + 4\pi^2 w^2)$	returns e^{-5t} ; at $t = -1$, e^5	$e^{-5 t }$; at $t = -1$, e^{-5}
$12/(36 + 4\pi^2 w^2)$	returns e^{-6t} ; at $t = -1$, e^6	$e^{-6 t }$; at $t = -1$, e^{-6}

Related Issues Assessment

The bug-finding harness performed a best-effort deduplication check and found no public issue, pull request, release-note entry, Stack Overflow post, or mailing-list thread describing this *specific* Lorentzian inverse transform returning $\exp(-t)$ instead of $\exp(-\text{Abs}(t))$. The underlying failure family, however, is already documented in the same `_fourier_transform` code path. SymPy [issue #22787](#) (*Bug in fourier_transform*, open, still reproducing on 1.14.0) reports `fourier_transform/_fourier_transform` returning a non-even result for a real even input, using the same evenness argument, and [issue #14624](#) (closed) reports `inverse_fourier_transform` giving a wrong answer on the negative half-line. The deduplication verdict is therefore *family known, specific instance new*: this is a new concrete instance of the known, still-open `_fourier_transform` Piecewise-branch failure family, not a novel bug. The case remains individually actionable because it gives a concrete, reproducible transform-pair counterexample and a narrow regression test, and the regression test may be linked to [issue #22787](#).

Proposed SymPy Regression Test

```
import pytest
from sympy import Abs, exp, inverse_fourier_transform, pi, simplify,
    symbols

@pytest.mark.parametrize("a", [1, 2, 3, 4, 5, 6])
def test_inverse_fourier_lorentzian_is_even(a):
    t, w = symbols("t w", real=True)
    expr = 2*a/(a**2 + 4*pi**2*w**2)
    inv = inverse_fourier_transform(expr, w, t)
    assert simplify(inv.subs(t, -1) - exp(-a*Abs(-1))) == 0
```

A.4.13 Bug 13: `nonlinsolve` returns a solution where the original rational equation is undefined

Minimal Reproducer

```
import sympy
from sympy import nonlinsolve, symbols

x, y = symbols("x y")
eqs = [(x*y - x)/(x - 1) - y, y - 1]
sol = nonlinsolve(eqs, [x, y])

print("SymPy version:", sympy.__version__)
print("nonlinsolve result =", sol)
```

```

for candidate in sol:
    subs = {x: candidate[0], y: candidate[1]}
    print("candidate =", candidate)
    print("residuals =", [eq.subs(subs) for eq in eqs])
print("denominator at (1, 1) =", (x - 1).subs(x, 1))

```

Observed Behavior

```

SymPy version: 1.14.0
nonlinsolve result = {(1, 1)}
candidate = (1, 1)
residuals = [nan, 0]
denominator at (1, 1) = 0

```

SymPy returns the concrete tuple (1,1). Direct substitution into the original first equation produces `nan`, because the denominator $x - 1$ vanishes at that point. Thus the returned tuple is not merely an alternate representation of a valid solution; it is outside the domain of the original system.

Expected Behavior

The mathematically correct solution set is `EmptySet`. A solver for rational equations must reject every candidate that makes any original denominator zero, even if that candidate solves the denominator-cleared polynomial equations.

Mathematical Explanation

The system submitted to `nonlinsolve` is

$$\frac{xy - x}{x - 1} - y = 0, \quad y - 1 = 0.$$

The second equation forces $y = 1$. Clearing the denominator in the first equation gives

$$xy - x - y(x - 1) = 0,$$

which simplifies to $y - x = 0$. Combining this denominator-cleared equation with $y = 1$ forces $x = 1$.

However, clearing the denominator is valid only under the domain condition $x \neq 1$. The only point produced by the cleared polynomial system is exactly the excluded point (1,1), where the original rational expression has denominator 0. Therefore no valid solution remains, and the correct result is the empty set.

Source-Level Diagnosis

The diagnosis identifies the relevant implementation in `sympy/solvers/solveset.py`, in the public `nonlinsolve` path. The traced call path is

`nonlinsolve` → `_separate_poly_nonpoly` → `_simple_dens` → `_handle_poly`,

followed by an early return in the zero-dimensional polynomial success path.

The helper `_separate_poly_nonpoly` records denominators before converting rational equations to numerators. For the reproducer, it records the denominator $x - 1$, then replaces the first rational equation by its numerator, which reduces to $-x + y$. The polynomial subsystem $[-x + y, y - 1]$ is then solved as $(x, y) = (1, 1)$.

The diagnosed bug is that when this polynomial subsystem solves all equations, `nonlinsolve` returns the polynomial solutions directly instead of applying the recorded denominator exclusions. The denominator-aware `substitution(..., exclude=denominators)` path is used only later, when remaining equations still need processing.

```
if not remaining:
    # If there is nothing left to solve then return the solution from
    # solve_poly_system directly.
    return FiniteSet(*map(to_tuple, poly_sol))
```

This control-flow choice causes the observed mathematical failure: the implementation has already recorded that $x - 1$ must be nonzero, but the early return bypasses that exclusion and admits the exact point where the denominator vanishes. A plausible fix direction supported by the diagnosis is to filter `poly_sol` against the recorded denominators before this early return, or to route the fully solved rational-polynomial case through the same denominator-aware validation used by `substitution`.

Additional Failing Instances

The independent verification checks the representative case and five shifted instances of the same family:

$$\left[\frac{xy - ax}{x - a} - y, y - a \right], \quad a \in \{1, 2, 3, 4, 5, 6\}.$$

For each a , the second equation forces $y = a$. The denominator-cleared first equation simplifies to $ay - ax = 0$, so the cleared system forces $x = a$. That is exactly the excluded value of the denominator $x - a$, so the expected result is empty throughout the family.

Input / Expression	SymPy behavior	Expected behavior
$a = 1 : \frac{xy-x}{x-1} - y = 0, y = 1$	Returns $\{(1, 1)\}$; $x - 1 = 0$.	EmptySet.
$a = 2 : \frac{xy-2x}{x-2} - y = 0, y = 2$	Returns $\{(2, 2)\}$; $x - 2 = 0$.	EmptySet.
$a = 3 : \frac{xy-3x}{x-3} - y = 0, y = 3$	Returns $\{(3, 3)\}$; $x - 3 = 0$.	EmptySet.
$a = 4 : \frac{xy-4x}{x-4} - y = 0, y = 4$	Returns $\{(4, 4)\}$; $x - 4 = 0$.	EmptySet.
$a = 5 : \frac{xy-5x}{x-5} - y = 0, y = 5$	Returns $\{(5, 5)\}$; $x - 5 = 0$.	EmptySet.
$a = 6 : \frac{xy-6x}{x-6} - y = 0, y = 6$	Returns $\{(6, 6)\}$; $x - 6 = 0$.	EmptySet.

Related Issues Assessment

The best-effort deduplication check found general background on extraneous solutions introduced by clearing denominators and identified the relevant public SymPy source context, but it did not find a public SymPy issue, pull request, Stack Overflow post, mailing-list discussion, or release note containing this exact system, the returned result $\{(1, 1)\}$, the residuals `[nan, 0]`, or the diagnosed bypass of `exclude=denominators`. The deduplication verdict is therefore a new failure mode with no public precedent. The bug remains individually actionable because it supplies a minimal reproducible system, a concrete invalid solution, a source-level control-flow explanation, and a concise regression test.

Proposed SymPy Regression Test

```
import pytest

from sympy import EmptySet, nonlinsolve, symbols

@pytest.mark.parametrize("a", [1, 2, 3, 4, 5, 6])
def test_nonlinsolve_rejects_zero_denominator_solution(a):
    x, y = symbols("x y")
    eqs = [(x*y - a*x)/(x - a) - y, y - a]
    assert nonlinsolve(eqs, [x, y]) == EmptySet
```

A.4.14 Bug 14: Rayleigh CDF is positive below support

Minimal Reproducer

```

from sympy.stats import P, Rayleigh, cdf

R = Rayleigh("R", 1)

print("cdf(R)(-1):", cdf(R)(-1))
print("P(R <= -1):", P(R <= -1))
print("expected cdf below support:", 0)
print("incorrect:", cdf(R)(-1) != 0)

```

Observed Behavior

```

cdf(R)(-1): 1 - exp(-1/2)
P(R <= -1): 0
expected cdf below support: 0
incorrect: True

```

The direct CDF call returns the positive expression $1 - \exp(-1/2)$ at a negative argument. This is inconsistent with the same distribution queried through $P(R \leq -1)$, which correctly returns 0.

Expected Behavior

For a Rayleigh random variable R with scale parameter $\sigma > 0$, $\text{cdf}(R)(z)$ should be 0 for every $z < 0$. In the representative case $R = \text{Rayleigh}(1)$, the mathematically correct result is therefore

$$\text{cdf}(R)(-1) = 0.$$

Mathematical Explanation

The Rayleigh distribution has support $[0, \infty)$. Its density is

$$f_{\sigma}(x) = \begin{cases} \frac{x}{\sigma^2} \exp\left(-\frac{x^2}{2\sigma^2}\right), & x \geq 0, \\ 0, & x < 0, \end{cases}$$

so its CDF is the corresponding piecewise function

$$F_{\sigma}(z) = \begin{cases} 0, & z < 0, \\ 1 - \exp\left(-\frac{z^2}{2\sigma^2}\right), & z \geq 0. \end{cases}$$

The closed form $1 - \exp(-z^2/(2\sigma^2))$ is only the nonnegative branch of the CDF. Extending that branch to negative z is not an equivalent representation: for $z < 0$, the event $R \leq z$ is empty because no Rayleigh-distributed value can

be negative. Thus the returned value is not a harmless simplification, branch choice, or alternate form; it assigns positive probability to a point below the distribution support.

Source-Level Diagnosis

The diagnosis identifies a support-guarding gap in the Rayleigh distribution’s custom CDF implementation. In `sympy/stats/crv_types.py`, `RayleighDistribution` declares the nonnegative support `Interval(0, ∞)`, but its `_cdf` method returns the unguarded nonnegative-branch formula $1 - \exp(-x^2/(2\sigma^2))$. The traced call path then reaches `SingleContinuousDistribution.cdf` in `sympy/stats/crv.py`. That method calls a distribution-specific `_cdf` first and returns it directly whenever it is not `None`, so the generic `compute_cdf` path that would account for support is bypassed.

```
class RayleighDistribution(SingleContinuousDistribution):
    set = Interval(0, oo)

    def _cdf(self, x):
        sigma = self.sigma
        return 1 - exp(-(x**2/(2*sigma**2)))

def cdf(self, x, **kwargs):
    if len(kwargs) == 0:
        cdf = self._cdf(x)
        if cdf is not None:
            return cdf
    return self.compute_cdf(**kwargs)(x)
```

This source-level behavior explains the observed failure: for $x = -1$, the Rayleigh-specific `_cdf` returns the algebraic expression $1 - \exp(-1/2)$, and the caller accepts it before any support condition can replace it by 0. A plausible fix direction, supported by the diagnosis, is to add the missing support guard to the Rayleigh CDF, for example by returning a piecewise expression whose below-support branch is 0, or otherwise ensuring that custom CDFs preserve the declared support.

Additional Failing Instances

The independent verification covers the representative case and five additional instantiations with positive scale parameters and negative evaluation points. In every row, $z < 0$, so $R \leq z$ is an empty event for a Rayleigh random variable. The expected result is therefore 0 across the family, regardless of the scale value.

Input / Expression	SymPy behavior	Expected behavior
$\sigma = 1, z = -1: \text{cdf}(R)(z)$	$1 - \exp(-1/2)$, while $P(R \leq z) = 0$	0
$\sigma = 2, z = -1: \text{cdf}(R)(z)$	$1 - \exp(-1/8)$, while $P(R \leq z) = 0$	0
$\sigma = 1, z = -2: \text{cdf}(R)(z)$	$1 - \exp(-2)$, while $P(R \leq z) = 0$	0
$\sigma = 3, z = -5: \text{cdf}(R)(z)$	$1 - \exp(-25/18)$, while $P(R \leq z) = 0$	0
$\sigma = 1/2, z = -1: \text{cdf}(R)(z)$	$1 - \exp(-2)$, while $P(R \leq z) = 0$	0
$\sigma = \sqrt{2}, z = -3: \text{cdf}(R)(z)$	$1 - \exp(-9/4)$, while $P(R \leq z) = 0$	0

Related Issues Assessment

The bug-finding harness performed a best-effort deduplication check and found a **novel** verdict, which should be framed as a new failure mode with no public precedent. The only related public material identified in the artifact is a discrete-support issue, SymPy issue #20031, which is related in theme because it concerns support handling, but it is not an exact duplicate of this Rayleigh CDF failure. The present bug remains individually actionable because it gives a concise reproducible case, a concrete affected distribution method, and a targeted regression test for the continuous Rayleigh CDF.

Proposed SymPy Regression Test

```
import pytest

from sympy import Rational, S, sqrt
from sympy.stats import Rayleigh, cdf

CASES = [
    (S.One, S.NegativeOne),
    (S(2), S.NegativeOne),
    (S.One, S(-2)),
    (S(3), S(-5)),
    (Rational(1, 2), S.NegativeOne),
    (sqrt(2), S(-3)),
]

@pytest.mark.parametrize("sigma,z", CASES)
def test_rayleigh_cdf_is_zero_below_support(sigma, z):
```

```
R = Rayleigh("R", sigma)
assert cdf(R)(z) == 0
```

A.4.15 Bug 15: Uniform Quantile Misses the $p = 1$ Endpoint

Minimal Reproducer

```
from sympy import FiniteSet, S
from sympy.stats import P, Uniform, cdf, quantile

U = Uniform("U", 0, 1)
actual = quantile(U)(S.One)
expected = FiniteSet(S.One)

print('quantile(Uniform("U", 0, 1))(1) =', actual)
print('cdf(Uniform("U", 0, 1))(1) =', cdf(U)(S.One))
print("P(U <= 1) =", P(U <= 1))
print("expected p=1 quantile =", expected)
print("matches expected =", actual == expected)
```

Observed Behavior

```
quantile(Uniform("U", 0, 1))(1) = EmptySet
cdf(Uniform("U", 0, 1))(1) = 1
P(U <= 1) = 1
expected p=1 quantile = {1}
matches expected = False
```

SymPy reports that the $p = 1$ quantile is the empty set, even though its own CDF and probability computations confirm that the uniform random variable reaches probability 1 at the upper endpoint.

Expected Behavior

For $U \sim \text{Uniform}(0, 1)$, the set-valued quantile API should return $\{1\}$:

$$\text{quantile}(U)(1) = \{1\}.$$

Equivalently, in SymPy's representation, the expected result is `FiniteSet(S.One)`.

Mathematical Explanation

SymPy documents the quantile as

$$Q(p) = \inf\{x \in \mathbb{R} : p \leq F(x)\},$$

where F is the CDF. For $U \sim \text{Uniform}(0, 1)$,

$$F(x) = \begin{cases} 0, & x < 0, \\ x, & 0 \leq x < 1, \\ 1, & x \geq 1. \end{cases}$$

At $p = 1$, the set $\{x : 1 \leq F(x)\}$ is $[1, \infty)$, so its infimum is 1. The result $\emptyset$ is therefore not an equivalent representation, a harmless branch choice, or a missing simplification: it says that no endpoint value satisfies the quantile request, contradicting the documented infimum definition and the computed facts $F(1) = 1$ and $P(U \leq 1) = 1$.

Source-Level Diagnosis

The diagnosis identifies the relevant generic implementation in `sympy/stats/crv.py`, specifically the `compute_quantile` method around lines 285–291. The public `quantile` function delegates to the random variable’s probability space distribution. The class-level `quantile` method around lines 296–302 first tries a distribution-specific `_quantile`, but `UniformDistribution` does not provide one, so the call falls back to `compute_quantile`. The documented quantile definition is in `sympy/stats/rv.py` around lines 1198–1203.

The generic implementation computes a CDF expression and then solves the equality $F(x) = p$ over the distribution support:

```
cdf = integrate(pdf, (x, left_bound, x), **kwargs)
quantile = solveset(cdf - p, x, self.set)
return Lambda(p, Piecewise((quantile, (p >= 0) & (p <= 1)), (nan, True)))
```

For `Uniform(0, 1)`, the traced generic computation produced:

```
Lambda(_p, Piecewise((Intersection({_p}, Interval.Ropen(0, 1)),
                      (_p >= 0) & (_p <= 1)), (nan, True)))
```

This equality-solving path represents the inverse CDF as an intersection with the right-open interval $[0, 1)$ in the trace, written by SymPy as `Interval.Ropen(0, 1)`. Substituting $p = 1$ therefore drops the endpoint and evaluates to $\emptyset$. The source-level behavior causes the mathematical failure because the quantile is not merely the solution set of $F(x) = p$; at an endpoint it must follow the documented infimum definition. A plausible fix direction supported by the diagnosis is to implement the generic quantile through the inequality-and-infimum definition, or to add endpoint handling after solving so that $p = 1$ returns the support supremum when the CDF at that endpoint is 1.

Additional Failing Instances

The independent verification checks the same endpoint property for six finite uniform intervals. It computes the uniform CDF directly using rational arithmetic and verifies that a point just below the upper endpoint has CDF less than

1, while the endpoint itself has CDF exactly 1. Thus each case has the same expected behavior: the $p = 1$ quantile is the interval's upper endpoint.

Input / Expression	SymPy behavior	Expected behavior
<code>Uniform(0, 1), p = 1</code>	$\emptyset$	$\{1\}$
<code>Uniform(-2, 3), p = 1</code>	$\emptyset$	$\{3\}$
<code>Uniform(2, 5), p = 1</code>	$\emptyset$	$\{5\}$
<code>Uniform(1/3, 7/3), p = 1</code>	$\emptyset$	$\{7/3\}$
<code>Uniform(-5/2, -1/2), p = 1</code>	$\emptyset$	$\{-1/2\}$
<code>Uniform(-1, 0), p = 1</code>	$\emptyset$	$\{0\}$

Related Issues Assessment

The best-effort deduplication check lists public Issue #21098 as related material and assigns the verdict `family_known_specific_new`. Based on the available evidence, this should be framed as a new instance of a known failure mode: the general endpoint fragility of quantile computation may be broadly understood, but this exact reproducible Uniform $p = 1$ endpoint case is previously unreported and remains individually actionable as a focused issue or regression test.

Proposed SymPy Regression Test

```
import pytest

from sympy import FiniteSet, Rational, S
from sympy.stats import Uniform, quantile

@pytest.mark.parametrize(
    "name,a,b",
    [
        ("unit_interval", S.Zero, S.One),
        ("shifted_interval", S(-2), S(3)),
        ("positive_interval", S(2), S(5)),
        ("rational_interval", Rational(1, 3), Rational(7, 3)),
        ("negative_interval", Rational(-5, 2), Rational(-1, 2)),
        ("ending_at_zero", S(-1), S.Zero),
    ],
)
def test_uniform_quantile_at_one_returns_upper_endpoint(name, a, b):
    U = Uniform("U_" + name, a, b)
    assert quantile(U)(S.One) == FiniteSet(b)
```

A.4.16 Bug 16: Laplace Transform of $\text{Heaviside}(a - t)$ Uses the Wrong Cutoff

Minimal Reproducer

```
from sympy import Heaviside, exp, integrate, laplace_transform, symbols

t = symbols("t", real=True)
s = symbols("s", positive=True)
a = symbols("a")

F = laplace_transform(Heaviside(a - t), t, s, noconds=True)
print(F)
print(F.subs(a, 2))
print(integrate(exp(-s*t), (t, 0, 2)))
```

Observed Behavior

```
laplace_transform(Heaviside(a - t), t, s, noconds=False)
# (exp(-a*s)/s, 0, (a > 0) & (a < 0) & Ne(1/a, 0))

laplace_transform(Heaviside(a - t), t, s, noconds=True)
# exp(-a*s)/s

laplace_transform(Heaviside(a - t), t, s, noconds=True).subs(a, 2)
# exp(-2*s)/s

integrate(exp(-s*t), (t, 0, 2))
# 1/s - exp(-2*s)/s
```

The unconditional result exposed by `noconds=True` specializes to $\exp(-2s)/s$ when $a = 2$. This is not equal to the direct defining integral over the interval where $\text{Heaviside}(2 - t)$ is nonzero.

Expected Behavior

For a positive cutoff a , SymPy should specialize the transform to

$$\mathcal{L}\{\text{Heaviside}(a - t)\}(s) = \int_0^a e^{-st} dt = \frac{1 - \exp(-as)}{s}.$$

In particular, after substituting $a = 2$, the expected result is

$$\frac{1 - \exp(-2s)}{s},$$

equivalently $1/s - \exp(-2s)/s$. For symbolic a , the result should preserve the relevant assumptions or conditions rather than returning an unconditional expression that becomes false under positive numeric specialization.

Mathematical Explanation

For $a > 0$, the function $\text{Heaviside}(a - t)$ is a closing step: it is one for $0 \leq t < a$ and zero for $t > a$. The value at the single endpoint $t = a$ does not affect the Laplace integral. Therefore

$$\int_0^\infty e^{-st} \text{Heaviside}(a - t) dt = \int_0^a e^{-st} dt = \frac{1 - \exp(-as)}{s} \quad (a > 0, s > 0).$$

The returned expression $\exp(-as)/s$ is the transform of an opening step at $t = a$, not of the finite window $[0, a]$. The discrepancy is not a harmless rewriting: for $a = 2$, the difference between SymPy's result and the defining integral is

$$\frac{\exp(-2s)}{s} - \left(\frac{1 - \exp(-2s)}{s} \right) = \frac{2\exp(-2s) - 1}{s},$$

which is not identically zero.

Source-Level Diagnosis

The diagnosis identifies the relevant implementation in `sympy/integrals/laplace.py`. The public call `laplace_transform` constructs a `LaplaceTransform` object. Its `doit` method dispatches to `_laplace_transform`, which attempts simple table rules before the programmatic Heaviside-specific handling.

The simple-rule table contains two relevant patterns for `Heaviside(a*t-b)`, one for an opening step and one for a closing step:

```
(Heaviside(a*t-b), exp(-s*b/a)/s, And(a > 0, b > 0), ...)
(Heaviside(a*t-b), (1-exp(-s*b/a))/s, And(a < 0, b < 0), ...)
```

For $\text{Heaviside}(a - t)$, this pattern matches with coefficient -1 and $b = -a$, so the mathematically relevant positive-window rule is the second one. However, `_laplace_apply_simple_rules` applies a table entry only when its condition is exactly true. With symbolic a , neither condition is exactly true, so this simple-rule path fails. The later transform path returns the tuple

$$\left(\frac{\exp(-as)}{s}, 0, (a > 0) \wedge (a < 0) \wedge \text{Ne}(1/a, 0) \right).$$

The expression component corresponds to the wrong opening-step branch, and the condition component is contradictory. The diagnosis further notes that `LaplaceTransform.doit` drops the condition when `noconds=True`, so the invalid expression $\exp(-as)/s$ becomes the public result. A plausible fix direction, supported by the diagnosis, is to return a guarded conditional or `Piecewise` transform for symbolic a , or otherwise avoid returning a formula whose contradictory condition can be discarded by `noconds=True`. The existing programmatic Heaviside branch already appears to encode the positive-window logic when the sign of the cutoff is known.

Additional Failing Instances

The verification covers the representative cutoff $a = 2$ and five additional positive cutoffs. In each case, SymPy first computes the same symbolic transform $\exp(-as)/s$ and then substitutes a positive integer cutoff. The expected answer has the same form across the family because, for every positive cutoff c , $\text{Heaviside}(c - t)$ restricts the Laplace integral to $[0, c]$.

Input / Expression	SymPy behavior	Expected behavior
$\text{Heaviside}(a - t)$, then $a = 2$	$\exp(-2s)/s$	$(1 - \exp(-2s))/s$
$\text{Heaviside}(a - t)$, then $a = 1$	$\exp(-s)/s$	$(1 - \exp(-s))/s$
$\text{Heaviside}(a - t)$, then $a = 3$	$\exp(-3s)/s$	$(1 - \exp(-3s))/s$
$\text{Heaviside}(a - t)$, then $a = 4$	$\exp(-4s)/s$	$(1 - \exp(-4s))/s$
$\text{Heaviside}(a - t)$, then $a = 5$	$\exp(-5s)/s$	$(1 - \exp(-5s))/s$
$\text{Heaviside}(a - t)$, then $a = 6$	$\exp(-6s)/s$	$(1 - \exp(-6s))/s$

Related Issues Assessment

The deduplication artifact lists PR #25755 and issue #24561 as related public material and gives the verdict `family_known_specific_new`. Framed in the requested terms, this is a new instance of a known failure mode: the broader area of Laplace-transform handling for Heaviside conditions may have public precedent, but this exact symbolic closing-step case and its positive-cutoff specializations are previously unreported in the available evidence. The bug therefore remains individually actionable as a concrete issue and regression test.

Proposed SymPy Regression Test

```
import pytest

from sympy import Heaviside, exp, laplace_transform, simplify, symbols

@pytest.mark.parametrize("cutoff", [2, 1, 3, 4, 5, 6])
def test_laplace_transform_symbolic_heaviside_closing_step_after_substitution(cutoff):
    t = symbols("t", real=True)
    s = symbols("s", positive=True)
    a = symbols("a")

    symbolic_transform = laplace_transform(Heaviside(a - t), t, s,
    noconds=True)
    actual = symbolic_transform.subs(a, cutoff)
```

```

expected = (1 - exp(-cutoff*s))/s

assert simplify(actual - expected) == 0

```

A.4.17 Bug 17: Telescoping Sum Ignores Undefined Terms

Minimal Reproducer

```

from sympy import Sum, oo, symbols

k = symbols("k", integer=True)
s = Sum(1/(k*(k - 1)), (k, 0, oo))

print("actual_sum =", s.doit())
print("first_terms =", [s.function.subs(k, i) for i in range(5)])
print("tail_sum_from_2 =", Sum(1/(k*(k - 1)), (k, 2, oo)).doit())

```

Observed Behavior

```

actual_sum = -1
first_terms = [zoo, zoo, 1/2, 1/6, 1/12]
tail_sum_from_2 = 1

```

SymPy returns the finite value -1 for a series whose first two terms are undefined. The output is not merely an alternative closed form for the same mathematical object: the requested series includes poles at the start of the summation range.

Expected Behavior

The call should not evaluate to the finite value -1 . The summand is undefined at $k = 0$ and $k = 1$, so the infinite sum from $k = 0$ is undefined in the ordinary sense. A correct result should preserve that undefinedness, for example by leaving the sum unevaluated or returning an explicit non-finite/undefined value rather than a finite number.

Mathematical Explanation

The summand has the partial-fraction form

$$\frac{1}{k(k-1)} = \frac{1}{k-1} - \frac{1}{k},$$

which telescopes only on ranges where all involved terms are defined. On the well-defined tail,

$$\sum_{k=2}^N \frac{1}{k(k-1)} = \sum_{k=2}^N \left(\frac{1}{k-1} - \frac{1}{k} \right) = 1 - \frac{1}{N},$$

and therefore

$$\sum_{k=2}^{\infty} \frac{1}{k(k-1)} = 1.$$

The requested sum, however, starts at $k = 0$. Its first two terms are $1/(0 \cdot -1)$ and $1/(1 \cdot 0)$, both undefined. Telescoping cannot cancel away terms that do not exist, so the finite value -1 is mathematically invalid.

Source-Level Diagnosis

The diagnosis identifies the faulty path in `sympy/concrete/summations.py`. The public call `Sum(...).doit()` reaches `Sum.doit`, which calls `eval_sum`. That then calls `eval_sum_symbolic`, which applies `apart`, splits the resulting addends, and invokes `telescopic` around `sympy/concrete/summations.py:1163-1171`.

For this summand, partial fractions produce $1/(k-1) - 1/k$. The telescoping logic recognizes $-1/k$ as a shifted negative copy of $1/(k-1)$ and accepts the telescoping endpoint formula. Endpoint construction in `telescopic_direct`, identified at `sympy/concrete/summations.py:946-967` (endpoint `Add` at line 967), returns an expression equivalent to evaluating L at the lower endpoint and R at infinity. For limits $(k, 0, \infty)$, this yields $-1 + 0 = -1$.

The source-level problem is that this shortcut assumes the summand and the decomposed telescoping terms are finite over the summation range. In this case, the original summand has poles at $k = 0$ and $k = 1$, and the partial-fraction terms expose exactly the singular pieces that the endpoint formula skips. The suggested fix direction in the diagnosis is to check for poles of the original summand in the summation range, especially finite initial terms of infinite sums, before accepting a telescoping result. If such singular terms are present, SymPy should avoid producing a finite telescoping value.

Additional Failing Instances

The independent verification exercises the shifted family

$$\sum_{k=s}^{\infty} \frac{1}{(k-s)(k-s-1)}$$

for $s \in \{0, 1, 2, 3, 4, 5\}$. In every case, the first two terms occur at $k = s$ and $k = s+1$, where the denominator is zero. The well-defined tail from $k = s+2$ has the same telescoping value 1, but the full requested series is undefined because it includes the singular initial terms.

Input / Expression	SymPy behavior	Expected behavior
$\sum_{k=0}^{\infty} \frac{1}{k(k-1)}$	returns finite value -1	undefined; poles at $k = 0, 1$
$\sum_{k=1}^{\infty} \frac{1}{(k-1)(k-2)}$	returns finite value -1	undefined; poles at $k = 1, 2$
$\sum_{k=2}^{\infty} \frac{1}{(k-2)(k-3)}$	returns finite value -1	undefined; poles at $k = 2, 3$
$\sum_{k=3}^{\infty} \frac{1}{(k-3)(k-4)}$	returns finite value -1	undefined; poles at $k = 3, 4$
$\sum_{k=4}^{\infty} \frac{1}{(k-4)(k-5)}$	returns finite value -1	undefined; poles at $k = 4, 5$
$\sum_{k=5}^{\infty} \frac{1}{(k-5)(k-6)}$	returns finite value -1	undefined; poles at $k = 5, 6$

Related Issues Assessment

The bug-finding harness performed a best-effort deduplication check. The closest public material found was SymPy documentation for telescoping and Gosper-style summation machinery, along with unrelated or different summation issues such as GitHub issues 12018 and 5229. The deduplication verdict is that this is a specific new instance of a known failure family: open SymPy issue [#8746](#) (“summation over poles of rational functions”) already documents the same failure mode of summation across poles returning a finite closed form, but this exact instance where `Sum.doit()` ignores singular initial terms of an infinite sum and returns a finite telescoping value is not separately reported. This remains individually actionable because the reproducer is concise, the affected summation path is identified, and the proposed regression test directly protects the problematic behavior.

Proposed SymPy Regression Test

```
import pytest

from sympy import Sum, oo, symbols

@pytest.mark.parametrize("shift", [0, 1, 2, 3, 4, 5])
def test_telescoping_sum_does_not_ignore_undefined_initial_terms(shift):
    k = symbols("k", integer=True)
    expr = 1/((k - shift)*(k - shift - 1))
    result = Sum(expr, (k, shift, oo)).doit()
    assert result.is_finite is not True
```

A.4.18 Bug 18: sinh Falsely Evaluates a Nonzero Split Imaginary-Period Argument to Zero

Minimal Reproducer

```
from sympy import I, pi, simplify, sinh

expr = sinh(I*pi + I*(pi - 1))
combined = sinh(I*(2*pi - 1))

print("sinh(I*pi + I*(pi - 1)) =", expr)
print("sinh(I*(2*pi - 1)) =", combined)
print("difference =", simplify(expr - combined))
```

Observed Behavior

```
sinh(I*pi + I*(pi - 1)) = 0
sinh(I*(2*pi - 1)) = -I*sin(1)
difference = I*sin(1)
cmath expected = -0.8414709848078966j
```

SymPy evaluates the split argument to zero even though the algebraically combined argument evaluates to $-I \sin(1)$. The recorded numerical oracle is also nonzero, so this is not merely a different but equivalent symbolic form.

Expected Behavior

The expression $\sinh(I\pi + I(\pi - 1))$ should evaluate to $-I \sin(1)$, with numerical value approximately $-0.8414709848078965 I$. Equivalently, it should agree with $\sinh(I(2\pi - 1))$.

Mathematical Explanation

The two arguments are identical:

$$I\pi + I(\pi - 1) = I(2\pi - 1).$$

For real t , the identity $\sinh(It) = I \sin(t)$ gives

$$\sinh(I(2\pi - 1)) = I \sin(2\pi - 1) = -I \sin(1).$$

Since $\sin(1) \neq 0$, the correct value is nonzero. The function $\sinh$ is entire, so there is no branch convention or missing integration constant that could make the split expression evaluate to zero while the combined expression evaluates to $-I \sin(1)$.

Source-Level Diagnosis

The diagnosis identifies the relevant source locations as follows.

File	Relevant function or helper
sympy/functions/elementary/hyperbolic.py	<code>sinh.eval</code> , lines 189–208
sympy/functions/elementary/trigonometric.py	<code>sin.eval</code> , lines 301–382
sympy/functions/elementary/trigonometric.py	<code>_peeloff_pi</code> , lines 154–159

The traced call path starts in `sinh.eval`. SymPy extracts the coefficient of I with `_imaginary_unit_as_coefficient` and rewrites the call as $I \sin(i_coeff)$. For the unevaluated split argument, that coefficient is the addend structure $(-1 + \pi) + \pi$. Then `sin.eval` calls `_peeloff_pi`.

The source-level issue is in `_peeloff_pi`. The helper iterates over `Add.make_args(arg)` and uses `a.coeff(pi)` to remove rational multiples of π . For the nested addend $-1 + \pi$, `a.coeff(pi)` is 1. The diagnosis reports that this increments the peeled π -coefficient but fails to preserve the non- π remainder -1 in the residual terms.

As a result, the argument $\pi + (\pi - 1)$ is treated as the exact multiple 2π . The downstream trigonometric simplification evaluates $\sin(2\pi)$ to zero, and the hyperbolic rewrite propagates this as $\sinh(I\pi + I(\pi - 1)) = I \cdot 0 = 0$. A plausible fix direction supported by the diagnosis is to peel a π multiple from an addend only when the whole addend is a pure rational multiple of π , or otherwise subtract the peeled multiple and keep any nonzero remainder in the residual terms.

Additional Failing Instances

The independent verification checks the family

$$\sinh(I\pi + I(\pi - k)), \quad k \in \{1, 2, 3, 4, 5, 6\}.$$

For every listed integer offset, the same identity applies:

$$\sinh(I\pi + I(\pi - k)) = \sinh(I(2\pi - k)) = I \sin(2\pi - k) = -I \sin(k).$$

The expected value is therefore nonzero for these integer offsets, but the dropped-remainder mechanism makes the split argument look like an exact 2π trigonometric argument.

Input / Expression	SymPy behavior	Expected behavior
$\sinh(I\pi + I(\pi - 1))$	0	$-I \sin(1)$
$\sinh(I\pi + I(\pi - 2))$	0	$-I \sin(2)$
$\sinh(I\pi + I(\pi - 3))$	0	$-I \sin(3)$
$\sinh(I\pi + I(\pi - 4))$	0	$-I \sin(4)$
$\sinh(I\pi + I(\pi - 5))$	0	$-I \sin(5)$
$\sinh(I\pi + I(\pi - 6))$	0	$-I \sin(6)$

Related Issues Assessment

The bug-finding harness performed a best-effort deduplication check. The closest related public issues involve trigonometric periodicity and the same `_peeloff_pi` helper: SymPy issue #12328 discusses `_peeloff_pi` in a assumptions-metadata case involving $\sin(m\pi + \pi)$, while issue #17976 concerns incomplete periodic simplification such as $\sin(2n\pi + 4)$. These are related at the subsystem level but do not report the exact split imaginary-period reproducer, the wrong zero result, or the dropped non- π remainder from $\pi - 1$. The deduplication verdict is therefore a new instance of a known failure mode, and the concrete reproducer remains individually actionable as a SymPy issue and regression test.

Proposed SymPy Regression Test

```
import pytest

from sympy import I, pi, simplify, sin, sinh

@pytest.mark.parametrize("offset", [1, 2, 3, 4, 5, 6])
def test_sinh_split_imaginary_period_preserves_remainder(offset):
    assert simplify(sinh(I*pi + I*(pi - offset)) + I*sin(offset)) == 0
```

A.4.19 Bug 19: rsolve returns a solution that violates supplied initial conditions

Minimal Reproducer

```
from sympy import Function, rsolve, symbols

n = symbols("n", integer=True)
y = Function("y")
```

```

rec = y(n + 2) - (n + 2)*y(n + 1) + y(n)
sol = rsolve(rec, y(n), {y(0): 1, y(1): 1})

print("rsolve output:", sol)
print("returned y(0):", sol.subs(n, 0))
print("returned y(1):", sol.subs(n, 1))

```

Observed Behavior

```

rsolve output: 0
returned y(0): 0
returned y(1): 0
manual first terms: [1, 1, 1, 2, 7, 33]

```

SymPy returns the zero sequence. Direct substitution into that returned expression gives $y(0) = 0$ and $y(1) = 0$, even though the call explicitly supplied $y(0) = 1$ and $y(1) = 1$. The result is therefore not merely an incomplete closed form or a different representation of the intended sequence; it violates the constraints passed to the public API.

Expected Behavior

The solver should return a sequence satisfying the recurrence

$$y(n + 2) - (n + 2)y(n + 1) + y(n) = 0$$

and the initial conditions $y(0) = 1$, $y(1) = 1$, or else report that it cannot solve the recurrence with those initial conditions. It should never return the zero sequence for this call.

Mathematical Explanation

The recurrence is equivalent to

$$y(n + 2) = (n + 2)y(n + 1) - y(n).$$

Starting from the supplied initial values gives

$$\begin{aligned}
y(0) &= 1, \\
y(1) &= 1, \\
y(2) &= 2y(1) - y(0) = 1, \\
y(3) &= 3y(2) - y(1) = 2, \\
y(4) &= 4y(3) - y(2) = 7, \\
y(5) &= 5y(4) - y(3) = 33.
\end{aligned}$$

Thus a valid solution with these initial conditions begins

$$1, 1, 1, 2, 7, 33, \dots$$

The zero sequence does satisfy the homogeneous recurrence, but it satisfies $y(0) = 0$ and $y(1) = 0$, not the supplied nonzero initial conditions. Hence SymPy's answer is a solution of the recurrence alone, not a solution of the initial-value problem requested by the user.

Source-Level Diagnosis

The diagnosis identifies the relevant implementation as `sympy/solvers/recurr.py`, in the public `rsolve` function around lines 816–841. The traced call path is `sympy.rsolve(...)` to coefficient extraction, then to `rsolve_hyper(coeffs, 0, n, symbols=True)` near line 806, followed by the initial-condition handling in `rsolve`. For the representative recurrence, the traced internal call `rsolve_hyper([1, -(n + 2), 1], 0, n, symbols=True)` returns the constant-free candidate 0 with an empty list of arbitrary constants.

The problematic rule is that initial conditions are only applied when the candidate expression contains symbols for arbitrary constants:

```
if symbols and init is not None:
    ...
```

When `symbols` is empty, this block is skipped. In the failing case `solution` is already the constant-free expression 0, so `rsolve` never forms or checks the equations `solution.subs(n, 0) - 1` and `solution.subs(n, 1) - 1`. It then returns the candidate unchanged. This source-level behavior causes the mathematical failure because a constant-free candidate can still contradict explicit initial conditions. The diagnosis supports a plausible fix direction: when initial conditions are provided, `rsolve` should validate constant-free solutions as well, returning `None` or the same unsolved initial-condition failure used elsewhere when any supplied condition is false.

Additional Failing Instances

The independent verification checks the family

$$y(n+2) - (n+a)y(n+1) + y(n) = 0, \quad y(0) = 1, \quad y(1) = 1,$$

for $a = 2, 3, 4, 5, 6, 7$. In every listed case SymPy returns the zero sequence, so the returned values at $n = 0$ and $n = 1$ are both 0. The expected behavior is the same across the family: the returned expression must satisfy the two supplied initial values, or the solver should report that it cannot produce such a solution.

Input / Expression	SymPy behavior	Expected behavior
$a = 2 :$ $y(n+2) - (n+2)y(n+1) + y(n) = 0,$ $y(0) = y(1) = 1$	returns 0; returned $y(0) = 0, y(1) = 0$	satisfy $y(0) = y(1) = 1;$ forward terms 1, 1, 1, 2, 7, 33
$a = 3 :$ $y(n+2) - (n+3)y(n+1) + y(n) = 0,$ $y(0) = y(1) = 1$	returns 0; returned $y(0) = 0, y(1) = 0$	satisfy $y(0) = y(1) = 1;$ forward terms 1, 1, 2, 7, 33, 191
$a = 4 :$ $y(n+2) - (n+4)y(n+1) + y(n) = 0,$ $y(0) = y(1) = 1$	returns 0; returned $y(0) = 0, y(1) = 0$	satisfy $y(0) = y(1) = 1;$ forward terms 1, 1, 3, 14, 81, 553
$a = 5 :$ $y(n+2) - (n+5)y(n+1) + y(n) = 0,$ $y(0) = y(1) = 1$	returns 0; returned $y(0) = 0, y(1) = 0$	satisfy $y(0) = y(1) = 1;$ forward terms 1, 1, 4, 23, 157, 1233
$a = 6 :$ $y(n+2) - (n+6)y(n+1) + y(n) = 0,$ $y(0) = y(1) = 1$	returns 0; returned $y(0) = 0, y(1) = 0$	satisfy $y(0) = y(1) = 1;$ forward terms 1, 1, 5, 34, 267, 2369
$a = 7 :$ $y(n+2) - (n+7)y(n+1) + y(n) = 0,$ $y(0) = y(1) = 1$	returns 0; returned $y(0) = 0, y(1) = 0$	satisfy $y(0) = y(1) = 1;$ forward terms 1, 1, 6, 47, 417, 4123

Related Issues Assessment

A re-audit of the deduplication check identifies open public issues that report the same underlying `rsolve_hyper-returns-0` failure family, in which `rsolve` silently returns the zero sequence for variable-coefficient / higher-order recurrences:

- [sympy/sympy#27902](#), “rsolve: Gives 0 for higher order recurrences” (open) — the most direct match: its example `rsolve(y(n+2) - n*y(n+1) - y(n), y(n))` returns 0, the same second-order variable-coefficient symptom.
- [sympy/sympy#17982](#), “Wrong result from rsolve” (open).
- [sympy/sympy#11063](#), “Some wrong answers from rsolve” (open).

The deduplication verdict is therefore *family known, specific instance new*: the broad `rsolve-returns-0` failure is publicly reported, while bug-019’s specific lens — the constant-free 0 candidate causing the `if symbols and init is not`

None initial-condition block to be skipped, so supplied conditions are never validated — is the new specific instance within that known family. The issue remains individually actionable because it has a compact reproducer, a localized source-level mechanism, and a direct regression test for the supported initial-condition API.

Proposed SymPy Regression Test

```
import pytest
from sympy import Function, S, rsolve, symbols

CASES = [2, 3, 4, 5, 6, 7]

@pytest.mark.parametrize("a", CASES)
def test_rsolve_constant_free_solution_satisfies_initial_conditions(a):
    n = symbols("n", integer=True)
    y = Function("y")
    rec = y(n + 2) - (n + a)*y(n + 1) + y(n)
    sol = rsolve(rec, y(n), {y(0): S.One, y(1): S.One})

    assert sol is not None
    assert sol.subs(n, 0) == S.One
    assert sol.subs(n, 1) == S.One
```

A.4.20 Bug 20: Conditional Uniform Density Keeps the Wrong Support

Minimal Reproducer

```
from sympy import S, integrate, oo, symbols
from sympy.stats import P, Uniform, density, given

x = symbols("x")
U = Uniform("U", 0, 1)
Y = given(U, U > S.Half)
d = density(Y)(x)

print("density:", d)
print("total mass:", integrate(d, (x, -oo, oo)))
print("mass below 1/2 from density:", integrate(d, (x, 0, S.Half)))
print("P(Y < 1/2):", P(Y < S.Half))
```

Observed Behavior

```

density: 2*Piecewise((1, (x >= 0) & (x <= 1)), (0, True))
total mass: 2
mass below 1/2 from density: 1
P(Y < 1/2): 0

```

SymPy applies the factor $2 = 1/P(U > 1/2)$, but the returned density is still supported on the original interval $[0, 1]$. As a result, the expression returned by `density` is not a probability density: it integrates to 2 and assigns mass 1 to $[0, 1/2]$, even though the conditional probability query gives $P(Y < 1/2) = 0$.

Expected Behavior

For $U \sim \text{Uniform}(0, 1)$ and $Y = U \mid U > 1/2$, the conditional density should be

$$f_Y(x) = \begin{cases} 2, & 1/2 < x \leq 1, \\ 0, & \text{otherwise.} \end{cases}$$

Equivalently, the returned density should have total mass 1, zero mass below the conditioning threshold, and value 0 at any probe point $x < 1/2$.

Mathematical Explanation

The conditional density is obtained by restricting the original density to the conditioning event and then renormalizing:

$$f_{U|U>1/2}(x) = \frac{f_U(x) 1_{\{x>1/2\}}}{P(U > 1/2)}.$$

Since $f_U(x) = 1$ on $[0, 1]$ and $P(U > 1/2) = 1/2$, this gives density 2 only on the restricted support $(1/2, 1]$.

SymPy's result instead represents $21_{[0,1]}(x)$. This is not an equivalent form of the conditional density: it is positive on points that violate the condition, and

$$\int_{-\infty}^{\infty} 21_{[0,1]}(x) dx = 2 \quad \text{while} \quad \int_{-\infty}^{\infty} f_{U|U>1/2}(x) dx = 1.$$

The discrepancy is therefore a support error and a normalization error, not a harmless simplification or alternate representation.

Source-Level Diagnosis

The diagnosis identifies the relevant implementation in `sympy/stats/crv.py`.

In that file, the relevant methods are `ContinuousPSpace.conditional_space`, around lines 446–462, and `ContinuousPSpace.compute_density`, around lines 333–340. The traced call path is

```

    sympy.stats.rv.given
→ ContinuousPSpace.conditional_space
    → Density.doit/density
→ ContinuousPSpace.compute_density.

```

For the reproducer, the conditional probability space has a `ConditionalContinuousDomain` with set `Interval.Lopen(1/2, 1)`, so the domain restriction itself is computed. The stored density, however, remains the original uniform density divided only by the conditioning normalizer.

```
domain = ConditionalContinuousDomain(self.domain, condition)
norm = domain.compute_expectation(self.pdf, **kwargs)
pdf = self.pdf / norm.xreplace(replacement)
density = Lambda(tuple(domain.symbols), pdf)
return ContinuousPSpace(domain, density)
```

This rule renormalizes the original density but does not multiply it by an indicator for the conditional domain. The second part of the failure occurs when `density(Y)` requests the density of the same variable. In that case, `compute_density` has no random symbols to marginalize, and `ConditionalContinuousDomain.compute_expectation` returns the stored expression unchanged when the variable list is empty. The condition $U > 1/2$ is therefore never applied to the returned density, producing $21_{[0,1]}$ instead of $21_{(1/2,1]}$.

The diagnosis suggests that a plausible fix is to ensure that conditional continuous densities include the condition's support, either by incorporating an indicator or `Piecewise` restriction in `conditional_space`, or by making density extraction for a `ConditionalContinuousDomain` apply the domain restriction even when no variables are marginalized. A general fix would need to avoid double-applying the original distribution support already present in many distribution PDFs.

Additional Failing Instances

The same failure occurs for a family of conditional uniform densities $Y_t = U \mid U > t$ with $0 < t < 1$. For every such threshold, the correct density is $1/(1-t)$ only on $(t, 1]$ and 0 below t . The verification cases check both global normalization and local support by integrating the returned density over $[0, t]$ and evaluating it at $t/2$.

Input / Expression	SymPy behavior	Expected behavior
$Y = \text{given}(U, U > 1/2), \text{density}(Y)(x)$	$21_{[0,1]}(x)$; total mass 2; mass on $[0, 1/2]$ is 1	$21_{(1/2,1]}(x)$; total mass 1; zero mass below $1/2$
$Y = \text{given}(U, U > 1/3), \text{density}(Y)(x)$	$\frac{3}{2}1_{[0,1]}(x)$; positive at $x = 1/6$	$\frac{3}{2}1_{(1/3,1]}(x)$; zero at $x = 1/6$
$Y = \text{given}(U, U > 1/4), \text{density}(Y)(x)$	$\frac{4}{3}1_{[0,1]}(x)$; positive at $x = 1/8$	$\frac{4}{3}1_{(1/4,1]}(x)$; zero at $x = 1/8$
$Y = \text{given}(U, U > 2/3), \text{density}(Y)(x)$	$31_{[0,1]}(x)$; positive at $x = 1/3$	$31_{(2/3,1]}(x)$; zero at $x = 1/3$
$Y = \text{given}(U, U > 3/4), \text{density}(Y)(x)$	$41_{[0,1]}(x)$; positive at $x = 3/8$	$41_{(3/4,1]}(x)$; zero at $x = 3/8$
$Y = \text{given}(U, U > 1/5), \text{density}(Y)(x)$	$\frac{5}{4}1_{[0,1]}(x)$; positive at $x = 1/10$	$\frac{5}{4}1_{(1/5,1]}(x)$; zero at $x = 1/10$

These are all instances of the same defect: the normalizing factor changes with t , but the returned support remains the unconditional support $[0, 1]$ instead of the conditional support $(t, 1]$.

Related Issues Assessment

The best-effort deduplication check found documentation and background material for `density`, `given`, and conditional probability distributions, but did not identify a public SymPy issue, pull request, release note, mailing list thread, or Stack Overflow post for this exact reproducer or the internal `ConditionalContinuousDomain` support-retention failure. The verdict is therefore a new failure mode with no public precedent. The bug remains individually actionable because it has a concise reproducer, a concrete invalid density, a localized source diagnosis, and a focused regression test.

Proposed SymPy Regression Test

```
import pytest

from sympy import S, integrate, oo, symbols
from sympy.stats import P, Uniform, density, given

@pytest.mark.parametrize("t", [S.Half, S(1)/3, S(1)/4, S(2)/3, S(3)/4, S(1)/5])
def test_conditional_uniform_density_support_is_restricted(t):
    x = symbols("x")
    U = Uniform("U", 0, 1)
    Y = given(U, U > t)
    d = density(Y)(x)

    assert integrate(d, (x, -oo, oo)) == 1
    assert integrate(d, (x, 0, t)) == 0
    assert d.subs(x, t/2) == 0
    assert P(Y < t) == 0
```